ITDS231
Lecture 5.1

Data Link Layer (Review+Address)
Error Detection/Correction

Asst. Prof. Dr. Thitinan Tantidham

Some slides from
- ITCS323/2021s1
- NETACAD Network Essentials/ITN7.02

มหาวิทยาลัยมหิดล
Mahidol University

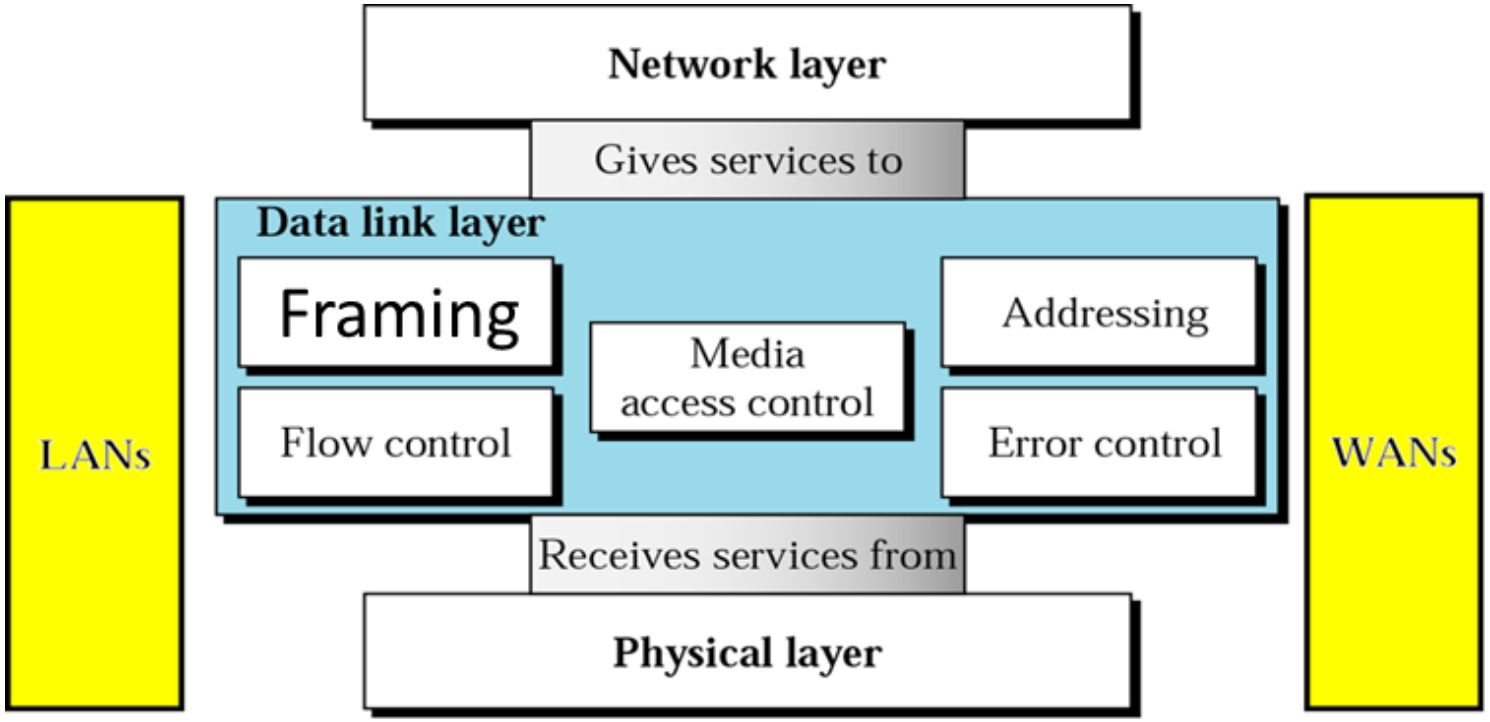
ห้ามมิให้นักศึกษาทำซ้ำ ดัดแปลง หรือใช้ประโยชน์จากงานอันมีลิขสิทธิ์ปรากฏอยู่ในระบบการศึกษาอิเล็กทรอนิกส์ (E-Learning) ของมหาวิทยาลัย ไม่ว่าจะทั้งหมดหรือบางส่วนโดยไม่ได้รับอนุญาตจากมหาวิทยาลัย นอกเหนือจากการศึกษาส่วนบุคคล ทั้งนี้ การทำซ้ำ ดัดแปลง หรือเผยแพร่ต่อสาธารณชนซึ่งงานอันมีลิขสิทธิ์ **จะมีโทษปรับตั้งแต่ 20,000 บาท ถึง 200,000 บาท และหากเป็นการกระทำเพื่อการค้า จะมีโทษจำคุกตั้งแต่ 6 เดือน ถึง 10 ปี หรือปรับตั้งแต่ 100,000 บาท ถึง 800,000 บาท หรือทั้งจำทั้งปรับ**

Unless allowed by Mahidol University, the User shall not copy, modify, or exploit in part or in whole of the copyrighted materials on the Platform, other than for your own individual study. The copy, modify or communication to public of the copyrighted materials shall be inflicted with a fine from 20,000 Baht up to 200,000 Baht. If the offence is committed with the commercial purpose, the offender shall be inflicted with imprisonment for a term from 6 months up to 10 years or a fine from 100,000 Baht up to 800,000 Baht or both imprisonment and fine.

Review: Data Link Layer

Review: The Data Link Layer

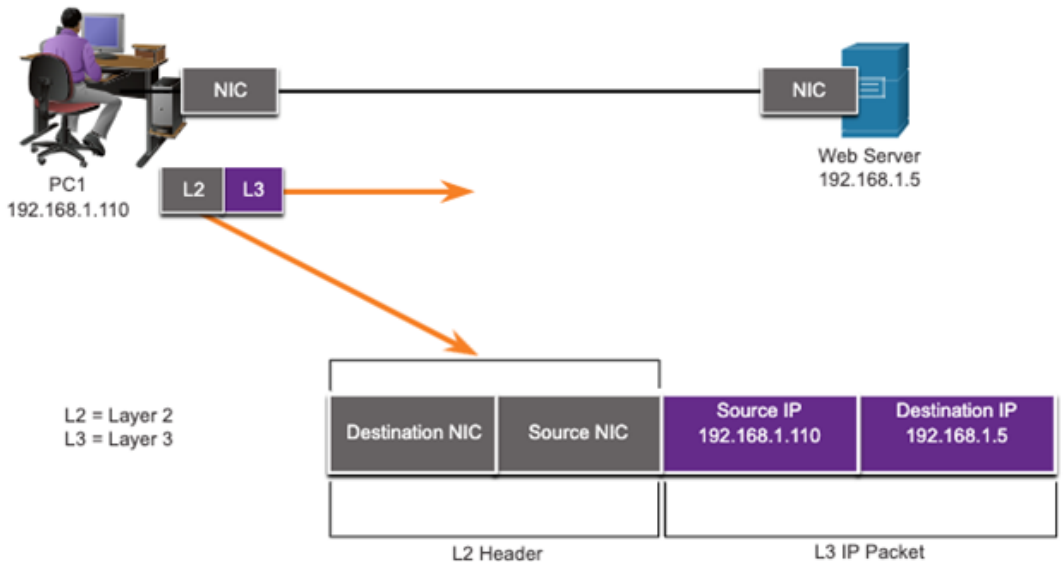
Purposes

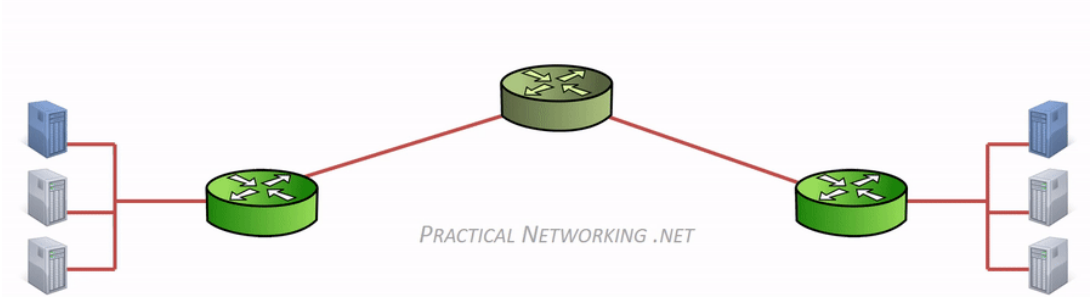


Review: The Data Link Layer

Purposes

- The Data Link layer is responsible for communications between end-device network interface cards.
- It allows upper layer protocols to access the physical layer media and encapsulates Layer 3 packets (IPv4 and IPv6) into Layer 2 Frames.
- It also performs error detection and rejects corrupts frames.
- It controls data flow between end-device network interface cards.
- Packets exchanged between nodes may experience numerous data link layers and media transitions.
- At each hop along the path, a router performs four basic Layer 2 functions:
  - ✓ Accepts a frame from the network medium.
  - ✓ De-encapsulates the frame to expose the encapsulated packet.
  - ✓ Re-encapsulates the packet into a new frame.
  - ✓ Forwards the new frame on the medium of the next network segment.





Review: Hub/Switch/Router

https://www.youtube.com/watch?v=1z0ULvg_pW8

https://www.youtube.com/watch?v=uuOytXJhuk0

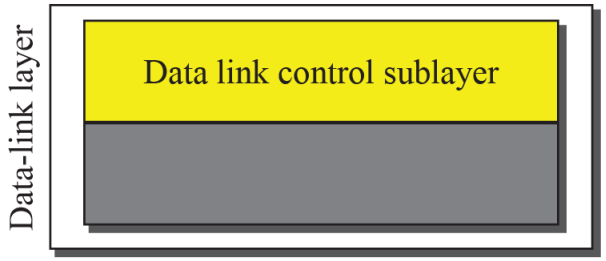
Review: The Data Link Sublayers

- The job of the data link layer is
  - ✓ **Error control**
  - ✓ **Flow control**
  - ✓ **Framing**
  - ✓ **Media access control**

  Error control, Flow control, Framing → Data/Logical Link Control sublayer

  Media access control → Media Access Control sublayer
- There are two sublayers:
  - ✓ Logical link control (LLC) sublayer
    - How stations control data flow with communication parties
  - ✓ Media access control (MAC) sublayer
    - How multiple stations access/share medium

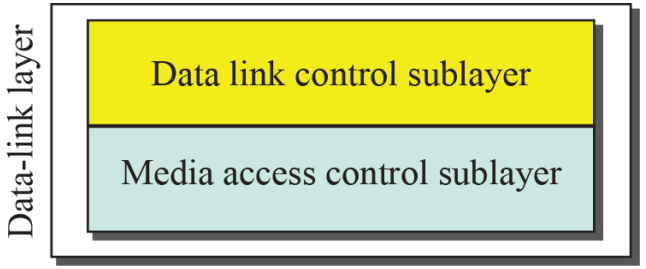
Point-to-Point (PPP)
High-Level Data Link Control (HDLC)
Frame-Relay



Communication for link in dedicated medium

WAN

Ethernet
802.11 Wireless

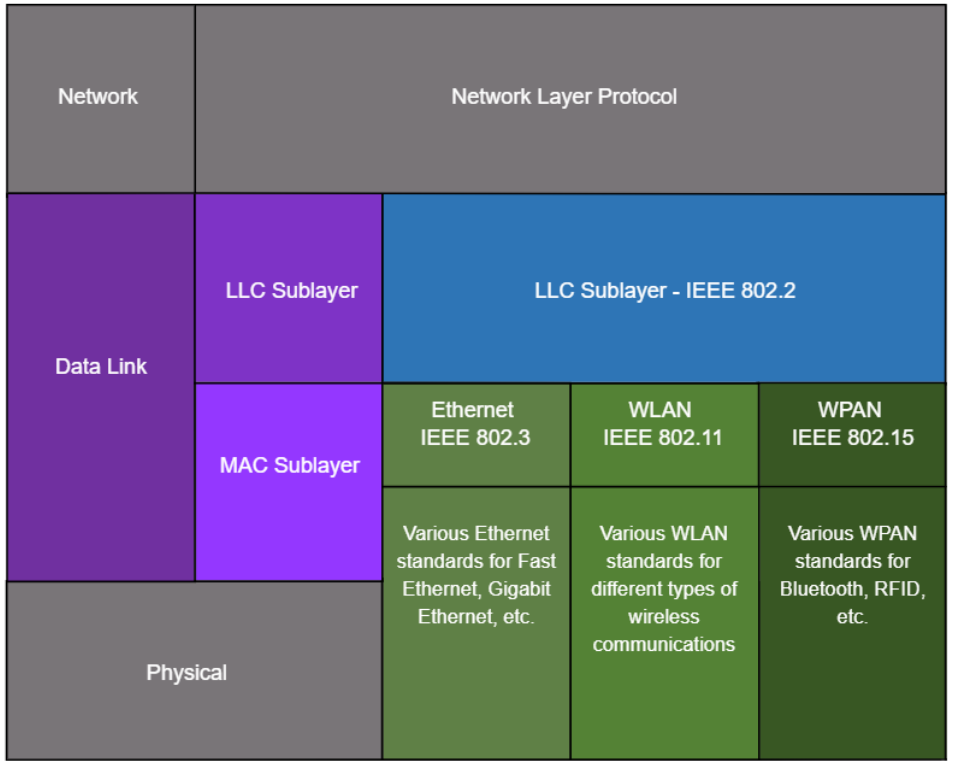


Communication for link in shared medium

LAN

Review: The Data Link Sublayers

IEEE 802 LAN/MAN



- IEEE 802 LAN/MAN standards are specific to the type of network (Ethernet, WLAN, WPAN, etc).
- The Data Link Layer consists of two sublayers. **Logical Link Control (LLC)** and **Media Access Control (MAC).**
  - The LLC sublayer communicates between the networking software at the upper layers and the device hardware at the lower layers.
  - The MAC sublayer is responsible for data encapsulation and media access control.

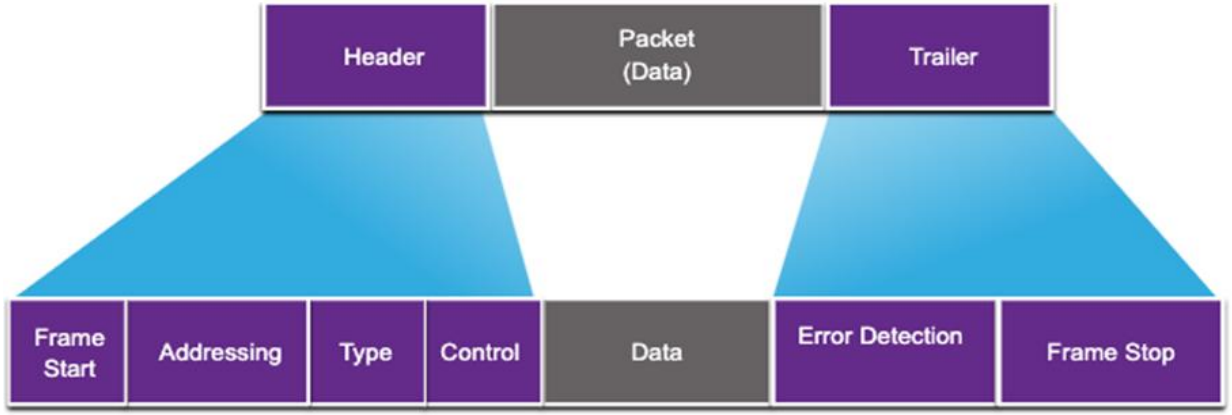
Review: The Data Link Layer Standards

IEEE 802 LAN/MAN

- Data link layer protocols are defined by engineering organizations:
  - ✓ Institute for Electrical and Electronic Engineers (IEEE).
  - ✓ International Telecommunications Union (ITU).
  - ✓ International Organizations for Standardization (ISO).
  - ✓ American National Standards Institute (ANSI).

Review: The Data Link Layer Frame

Frame Fields

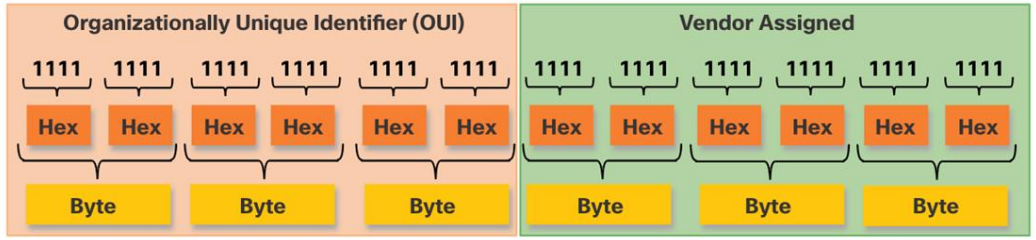


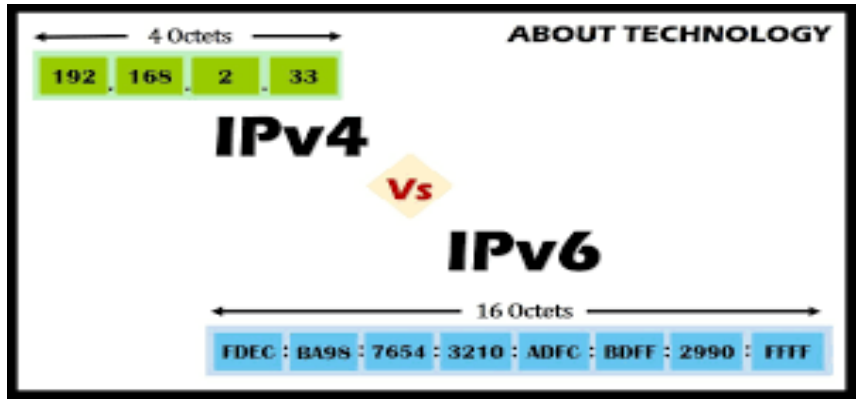
Field	Description
Frame Start and Stop	Identifies beginning and end of frame
Addressing	Indicates source and destination nodes
Type	Identifies encapsulated Layer 3 protocol
Control	Identifies flow control services
Data	Contains the frame payload
Error Detection	Used for determine transmission errors

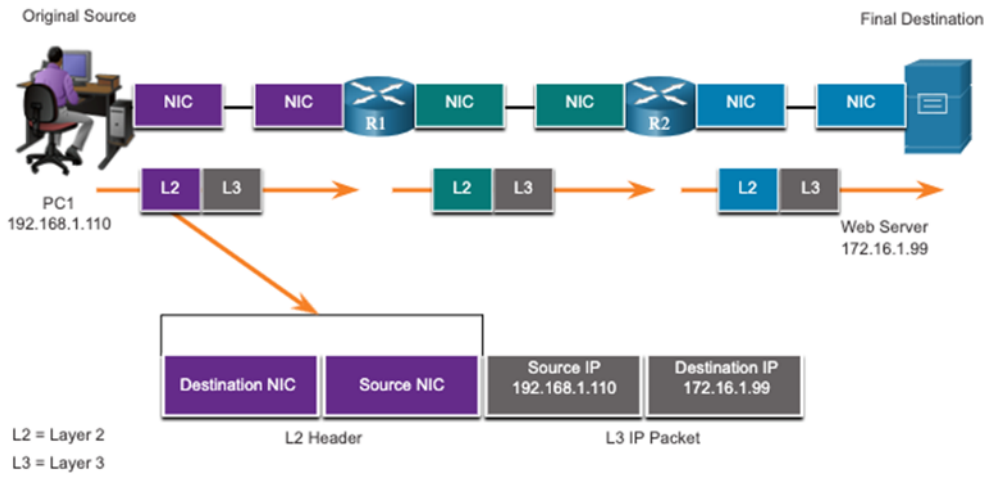
- Data is encapsulated by the data link layer with a header and a trailer to form a frame.
- A data link frame has three parts:
  - Header
  - Data
  - Trailer
- The fields of the header and trailer vary according to data link layer protocol.
- The amount of control information carried with in the frame varies according to access control information and logical topology.

Review: The Data Link Addresses

- Also referred to as a physical address.
- Contained in the frame header.
- Used only for local delivery of a frame on the link.
- Updated by each device that forwards the frame.







Decimal	Binary	Hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Frame Processing with MAC Address

Some slides from
- ITCS323/2021s1
- NETACAD Network Essentials/ITN7.02

Ethernet MAC Addresses

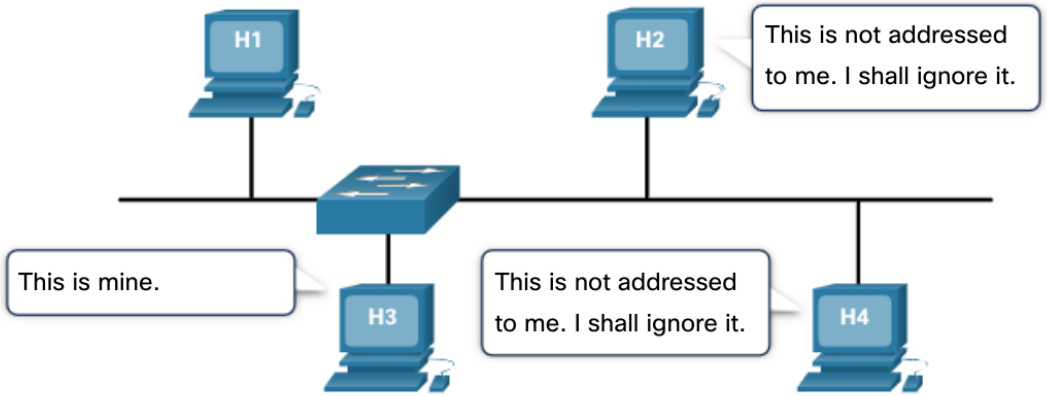
- When a device is forwarding a message to an Ethernet network, the Ethernet header include a Source MAC address and a Destination MAC address.

- When a NIC receives an Ethernet frame, it examines the destination MAC address to see if it matches the physical MAC address that is stored in RAM. If there is no match, the device discards the frame. If there is a match, it passes the frame up the OSI layers, where the de-encapsulation process takes place.

Note: Ethernet NICs will also accept frames if the destination MAC address is a broadcast or a multicast group of which the host is a member.

- Any device that is the source or destination of an Ethernet frame, will have an Ethernet NIC and therefore, a MAC address. This includes workstations, servers, printers, mobile devices, and routers.
- In Ethernet, different MAC addresses are used for Layer 2 unicast, broadcast, and multicast communications.

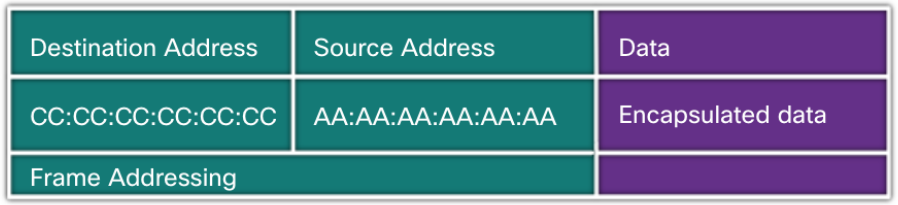
Destination Address	Source Address	Data
CC:CC:CC:CC:CC:CC	AA:AA:AA:AA:AA:AA	Encapsulated data
Frame Addressing		



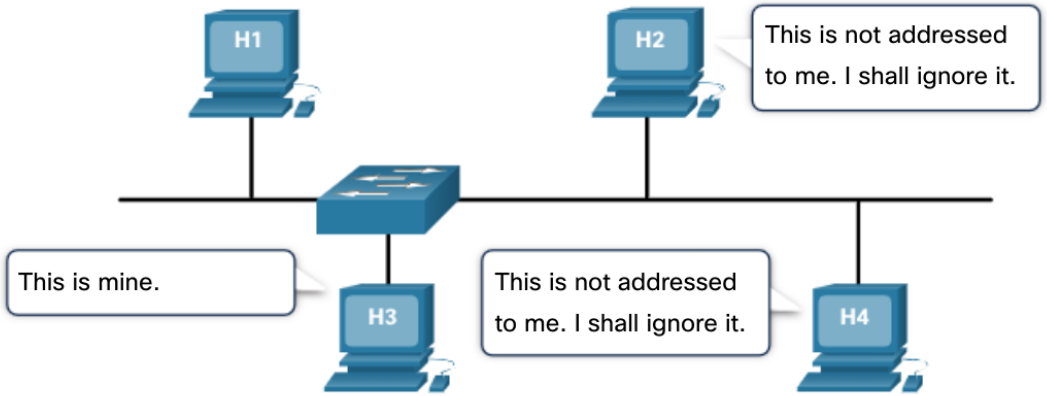
Ethernet MAC Addresses: Unicast MAC Address

- A unicast MAC address is the unique address that is used when a frame is sent from a single transmitting device to a single destination device.
- The process that a source host uses to determine the destination MAC address associated with **an IPv4** address is known as **Address Resolution Protocol (ARP).**
- The process that a source host uses to determine the destination MAC address associated with **an IPv6 address** is known **as Neighbor Discovery (ND).**

Note: The source MAC address must always be a unicast.

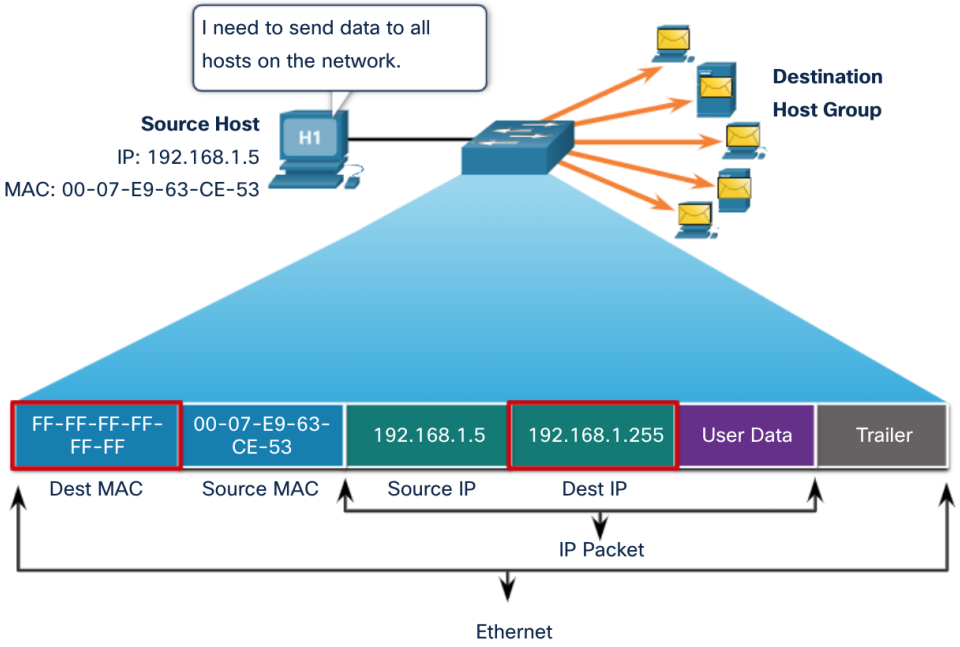


Destination Address	Source Address	Data
CC:CC:CC:CC:CC:CC	AA:AA:AA:AA:AA:AA	Encapsulated data
Frame Addressing		



Ethernet MAC Addresses: Broadcast MAC Address

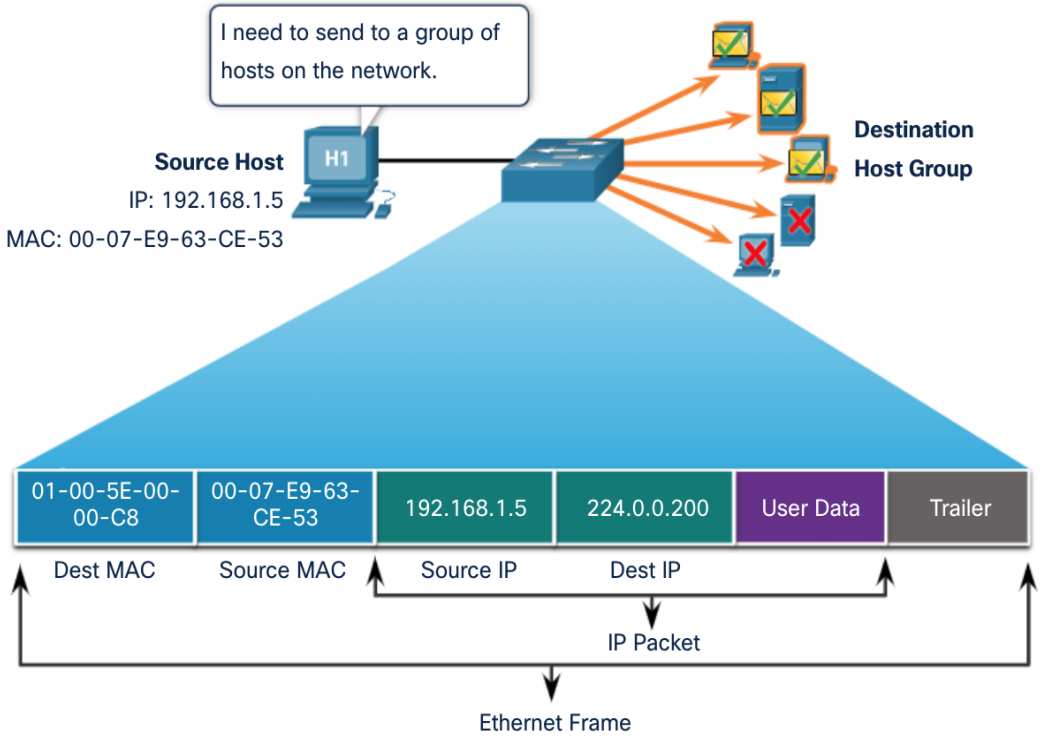
- An Ethernet broadcast frame is received and processed by every device on the Ethernet LAN. The features of an Ethernet broadcast are as follows:
- It has a destination MAC address of **FF-FF-FF-FF-FF-FF** in hexadecimal (48 ones in binary).
- It is flooded out all Ethernet switch ports except the incoming port. It is not forwarded by a router.
- If the encapsulated data is an IPv4 broadcast packet, this means the packet contains a destination IPv4 address that has all ones (1s) in the host portion. This numbering in the address means that all hosts on that local network (broadcast domain) will receive and process the packet.



Ethernet MAC Addresses: Multicast MAC Address

•An Ethernet multicast frame is received and processed by a group of devices that belong to the same multicast group.

- There is a destination MAC address of **01-00-5E** when the encapsulated data is an **IPv4** multicast packet and a destination MAC address of **33-33** when the encapsulated data is **an IPv6** multicast packet.
- There are other reserved multicast destination MAC addresses for when the encapsulated data is not IP, such as Spanning Tree Protocol (STP).
- It is flooded out all Ethernet switch ports except the incoming port, unless the switch is configured for multicast snooping. It is not forwarded by a router, unless the router is configured to route multicast packets.
- Because **multicast addresses represent a group of addresses** (sometimes called a host group), they can only be used as the destination of a packet. The source will always be a unicast address.
- As with the unicast and broadcast addresses, **the multicast IP address requires a corresponding multicast MAC address.**

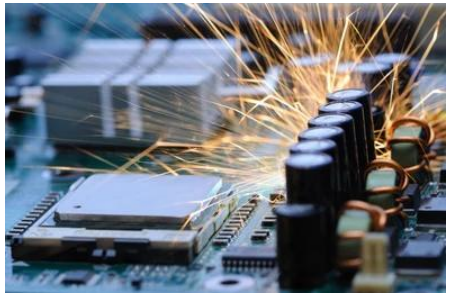


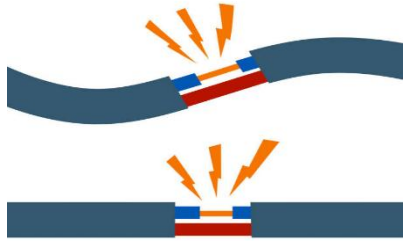
Error Detection

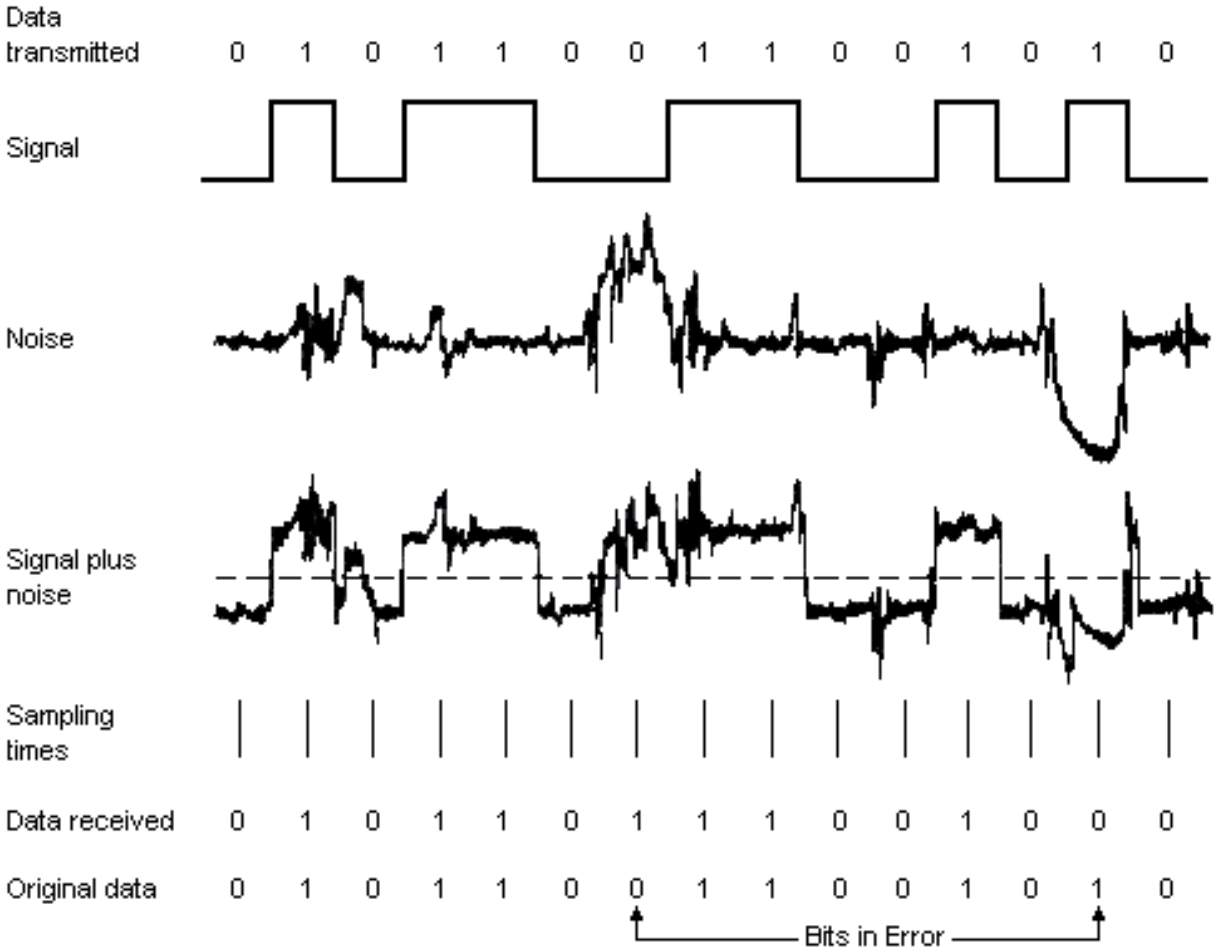
Some slides from
- ITCS323/2021s1
- NETACAD Network Essentials/ITN7.02

Bit Error During Transmission

- Bit error occurs mostly due to noise
  - Signal interference
  - Thermal noise
  - Hardware malfunction (device & cable problem)
  - Natural phenomenon (e.g. lightning, thunder)
  - Warfare (e.g., EMP (Electromagnetic Pulse) shockwave)





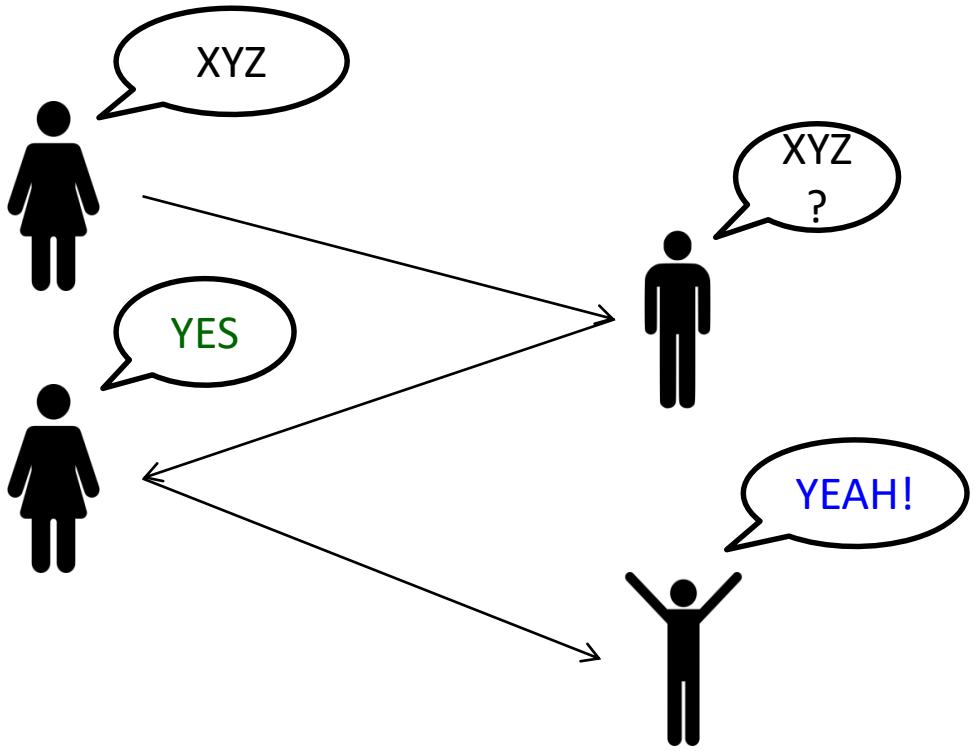


8-bit burst error

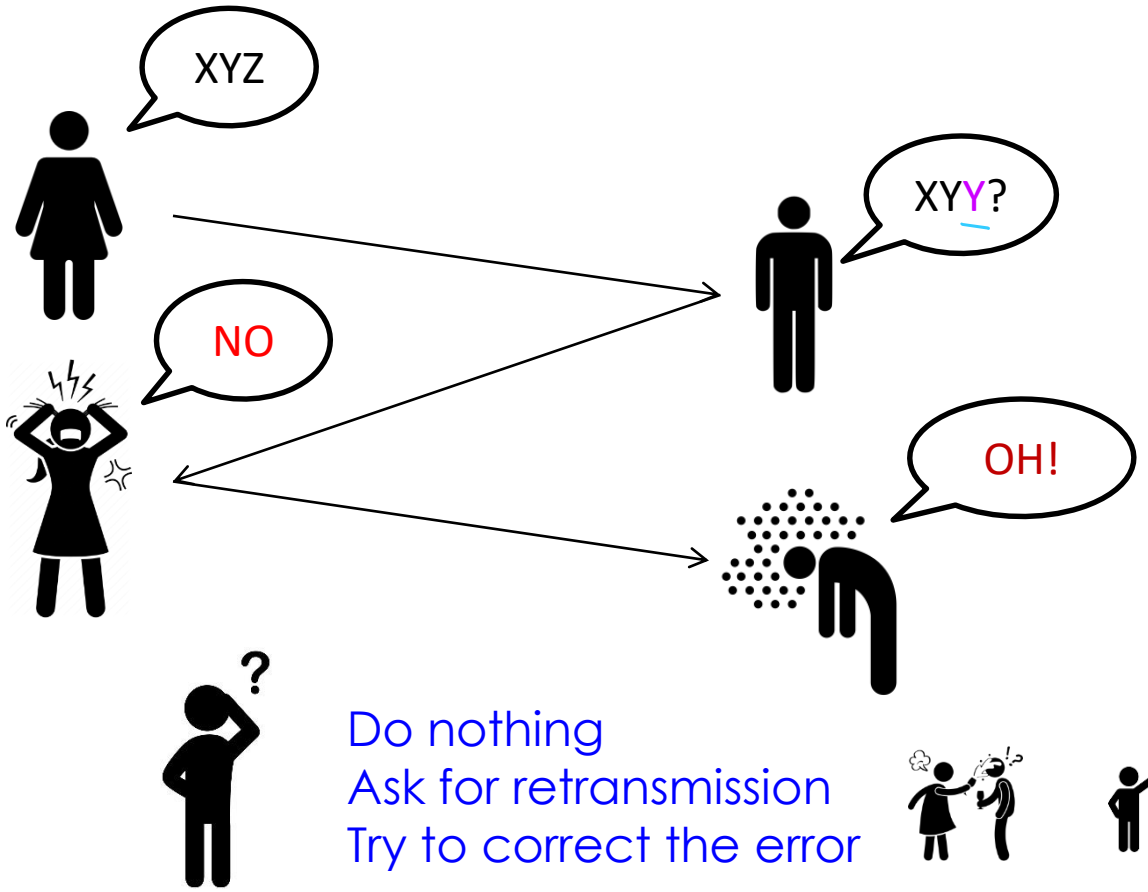
Basic Human Error Control

Human checks for error by asking the sender again for information verification.

No Error



With Error



Do nothing
Ask for retransmission
Try to correct the error

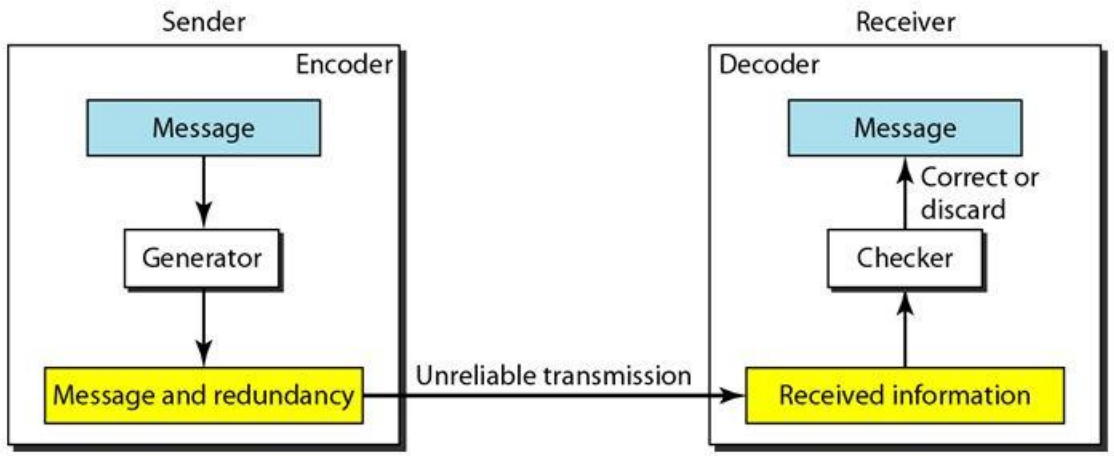
Error Control

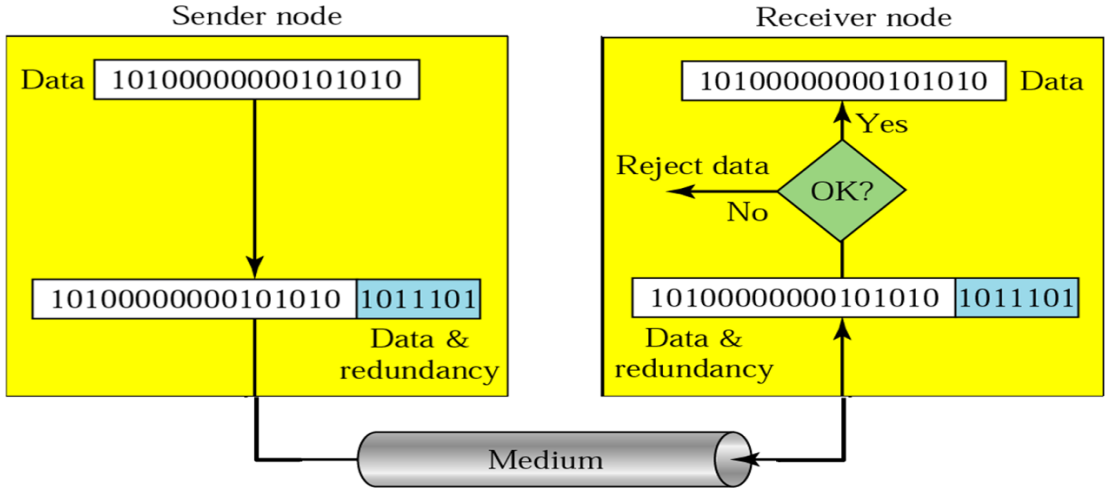
Error control in the data link layer is based on automatic repeat request (ARQ), which is the retransmission of data.

- Error control is both error detection and error correction
- Error control in the data link layer is often implemented simple
  - ✓ Detection and Retransmission
  - ✓ Called Automatic Repeat Request (ARQ)

Redundancy

- For computer data communication, it would be more efficient if the receiver knows whether the information received is in error without asking the sender every time.
- In order to do this, the sender must provide some hints to help the receiver verifies the correctness or integrity of data.
- Error detection uses the concept of redundancy, which means adding extra bits for detecting errors at the destination. they are discarded as soon as the accuracy of the transmission has been determined



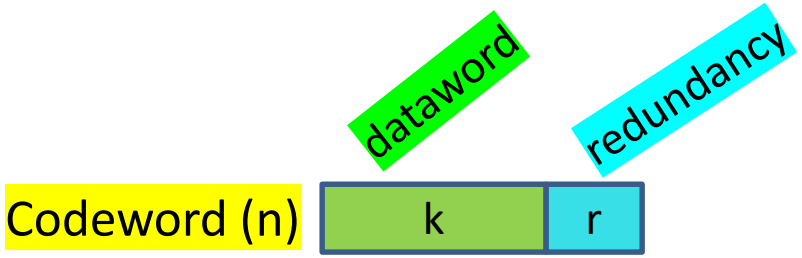


Block Coding

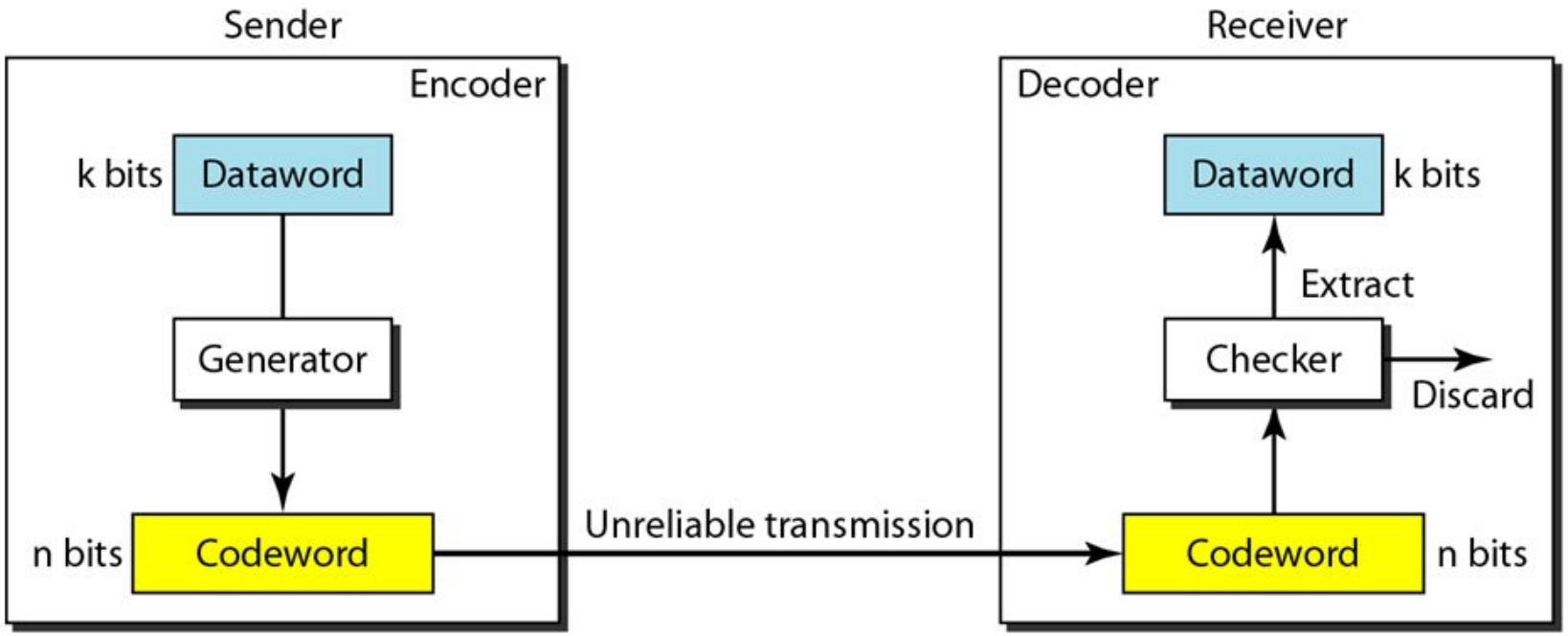
- When we want to add extra redundancy, we need to create the redundancy that is related to the data so that we can be sure that the redundancy can help us check the validity of the data.
- Since it is not reasonable to create redundancy after the whole amount of data is transmitted. We shall divide data into **blocks** and send one block at a time along with redundancy to periodically check for error.

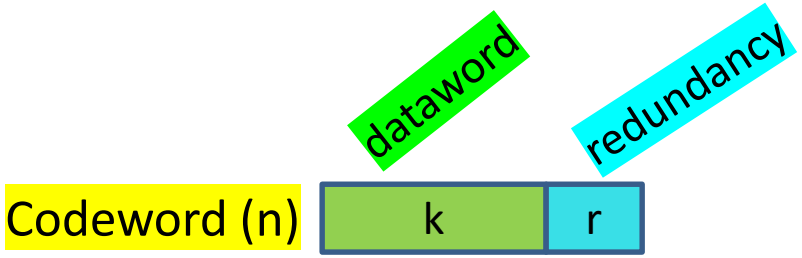
- If each block has **k** bits, there could be $\mathbf{2^k}$ types of dataword (k bits).
- The redundancy block has **r** bits and **r** should be smaller than **k** (otherwise, it wouldn't make any sense).
- The size of the codeword **n** = **k** + **r**. There could be $\mathbf{2^n}$ types of codeword (**n** bits) for **k**-bit datawords.

- Please note that while there could be $\mathbf{2^k}$ types of datawords, there are usually less than $\mathbf{2^n}$ codewords in use since we may not use all possible values of redundancy bits.



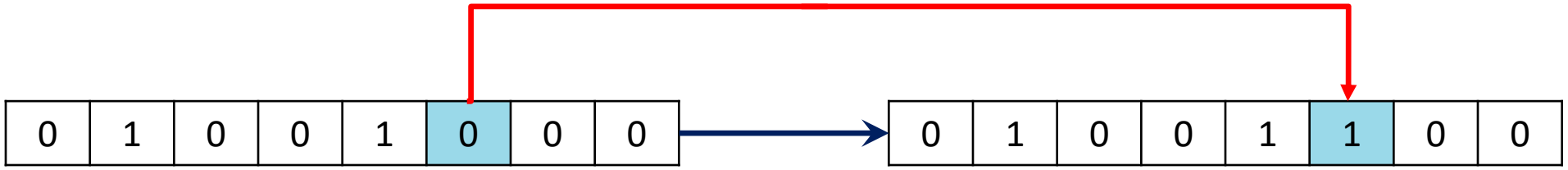
Block Coding



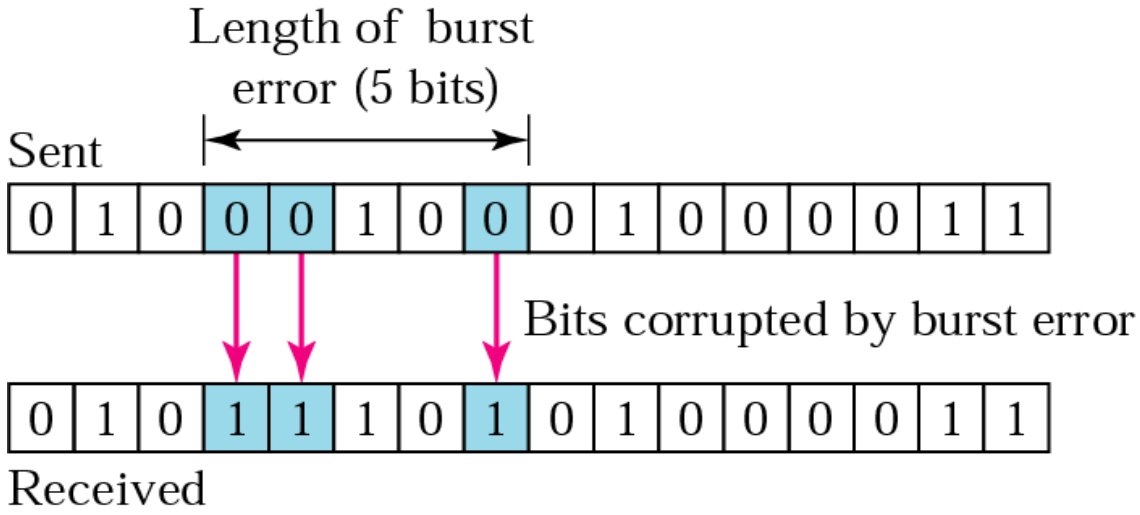


Errors in Computer Data Communication

- Single Bit Error



- Burst Error



How many are Bit Errors?

Example

- If the sender transmits with the rate of 2 Mbps, and the noise length is 10 microseconds. How many bits will be in error?

Solution:

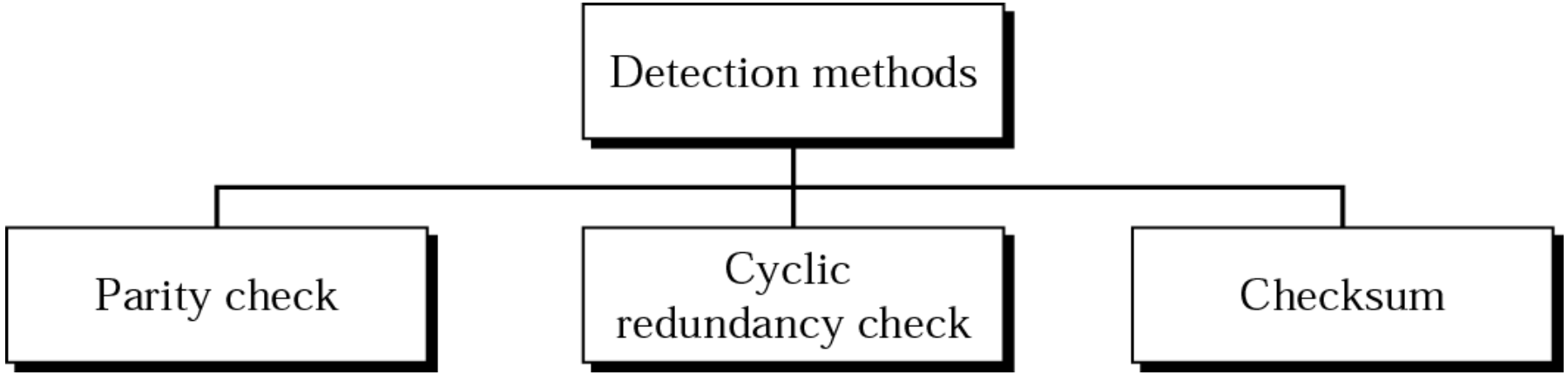
✓ Bit rate = 2 Mbps.

✓ 1 second = 2 Mbits

✓ 10 microsecond (μs) = 2 Mbits x (10×10^{-6}) = 20 bits

Error Detection

- Error Detection is simpler than error correction and is the first step in error correction process.

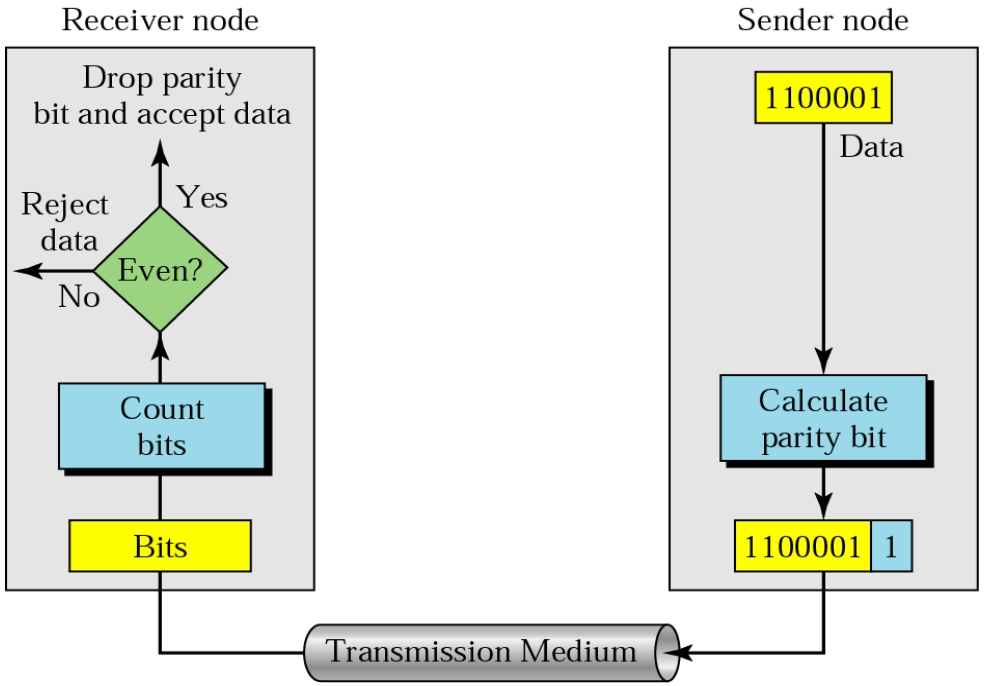


Parity Check

- **Even parity**
  - Redundancy bits are added to ensure that the total number of 1's in the codeword is even.
- **Odd parity**
  - Redundancy bits are added to ensure that the total number of 1's in the codeword is odd.

In parity check, a parity bit is added to every data unit so that the total number of 1s is even for even-parity (or odd for odd-parity).

Even Parity Bit Checking



Syndrome

Even parity

- When the receiver performs the same principle function as the sender, the result of the operation is called a syndrome.

- If the syndrome is 0, the receiver will assume that there is no error.

- Otherwise, the receiver will assume that there is an error.

0	1	0	0	1	0	0	1	1

0	1	0	0	1	0	0	1	1

Syndrome = 0+1+0+0+1+0+0+1+1 = **0** (half-adder)

0	1	0	0	1	0	0	1	1

0	1	0	0	1	0	1	1	1

Syndrome = 0+1+0+0+1+0+**1**+1+1 = **1** (half-adder)

Even Parity Bit Checking

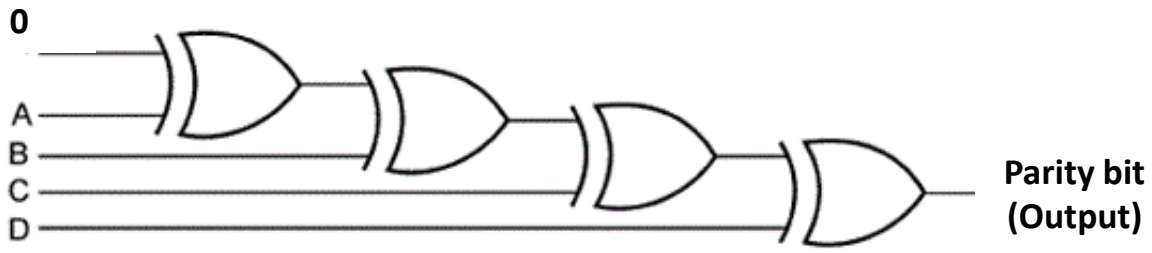
Example: byte (character) oriented: ASCII (7bits), EBCDIC(8bits)

- Suppose the sender wants to send the word world. In ASCII the five characters are coded as
- 1110111 1101111 1110010 1101100 1100100
- The following shows the actual bits sent
- 11101110 11011110 11100100 11011000 11001001
- Now suppose the word world in Example 1 is corrupted during transmission. (syndrome =1→error bits)
- 11111110 11011110 11101100 11011000 11001001
- The receiver would know about the error.

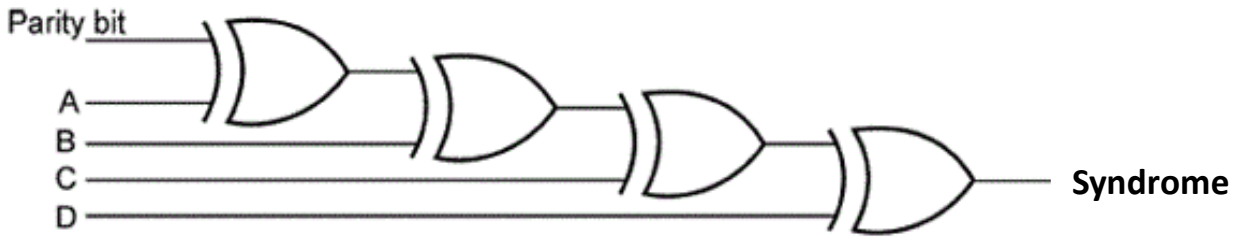
Parity Bit Generator and Checker

- Typically the generator and checker are based on the same engine.
- The output of the checker is called a syndrome.

Generator



Checker



Even Parity Bit Checking: Limitation (ข้อจำกัด)

- Simple parity check can detect all single-bit errors. It can detect burst errors only if the total number of errors in each data unit is **odd**.

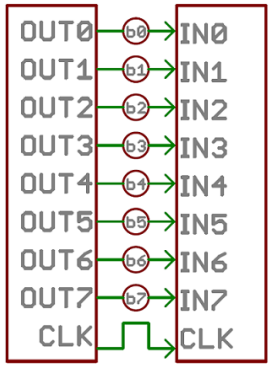
- Example: (even parity)
  - Actual sent: 11011011 10010011 01101100
  - Received: 11011011 11011011 01101100

- Can the receiver detect an error?

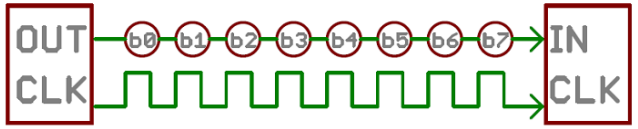
เราไม่สามารถตรวจสอบ error ได้
ถึงแม้มี bit ที่ error
Parity ไม่สามารถตรวจสอบกรณีที่มีจำนวน bits ที่เกิด error ที่เป็นเลขคู่

ตัวอย่างการใช้ Parity Bit

- Start Bit
- Data Bits
- Parity Bit (0/1)
- Stop Bit(s)



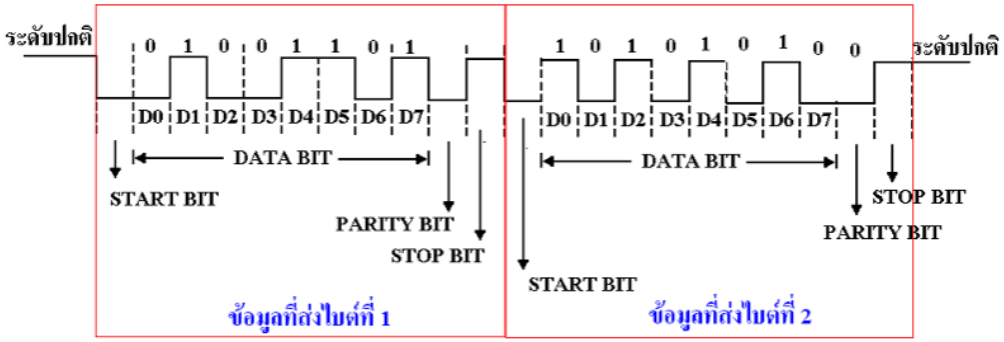
parallel



serial

8N2: 1 start bit, 8 data bits, 2 stop bits
8E1: 1 start bit, 8 data bits, 1 Even parity bit, 1 stop bit
8O1: 1 start bit, 8 data bits, 1 Odd parity bit, 1 stop bit

ตัวอย่างของ 8O1 หรือ 8E1



ASCII to Hexadecimal Conversion Table

MSB / LSB	0	1	2	3	4	5	6	7
0	NUL	DLE	SP	0	@	P	`	p
1	SOH	DC1	!	1	A	Q	a	q
2	STX	DC2	"	2	B	R	b	r
3	ETX	DC3	#	3	C	S	c	s
4	EOT	DC4	$	4	D	T	d	t
5	END	NAK	%	5	E	U	e	u
6	ACK	SYN	&	6	F	V	f	v
7	BEL	ETB	`	7	G	W	g	w
8	BS	CAN	(	8	H	X	h	x
9	HT	EM	)	9	I	Y	i	y
A	LF	SUB	*	:	J	Z	j	z
B	VT	ESC	+	;	K	[	k	{
C	FF	FS	,	<	L	\	l	|
D	CR	GS	-	=	M	]	m	}
E	SO	RS	.	>	N	^	n	~
F	SI	US	/	?	O	_	o	DEL

A = 0x41 = 0100 0001
ถ้า D0 เป็น LSB แล้ว D0 = 1, ..., D7 = 0
ถ้า D0 เป็น MSB แล้ว D0 = 0, ..., D7 = 1

I=0x49
=0100 1001

8E1

0100 10011

c=0x63
= 0110 0011

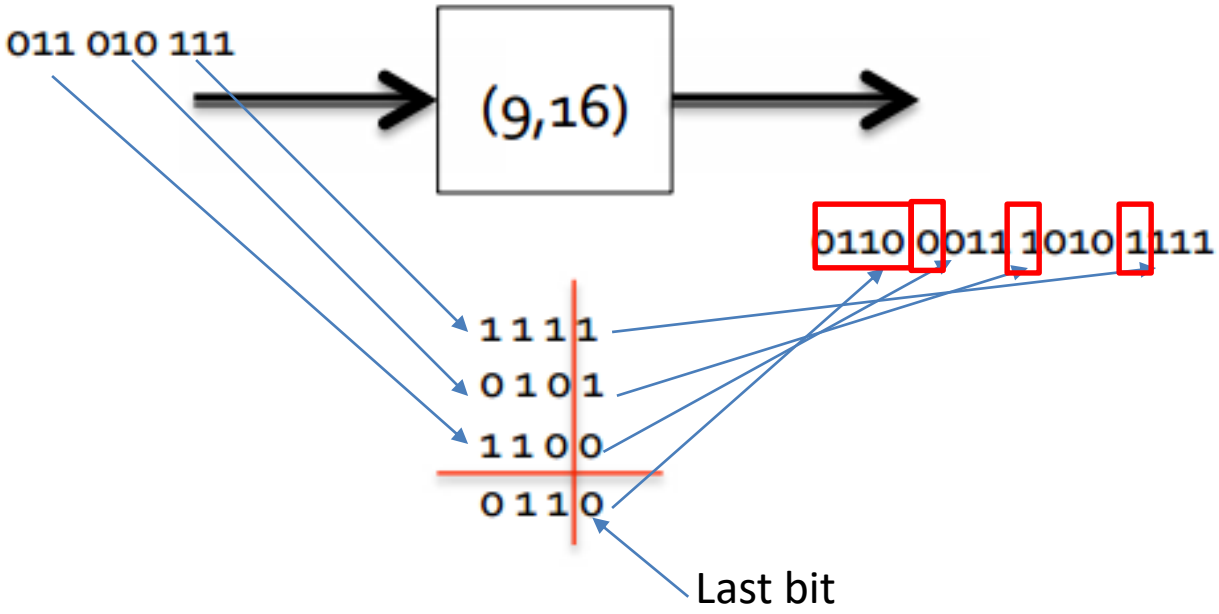
8E1
0110 00110

https://learn.sparkfun.com/tutorials/serial-communication/all

Two-Dimensional Parity Bit Checking (1/2)

- We can perform parity bit checking on both row and column in order to add an extra dimension of parity bit checking for burst error or even number of errors.
- Error correction capability. The erroneous position can be identified and fix the error.

Even parity
ส่งจากซ้ายไปขวา
ส่ง bit ขวาสุดก่อน



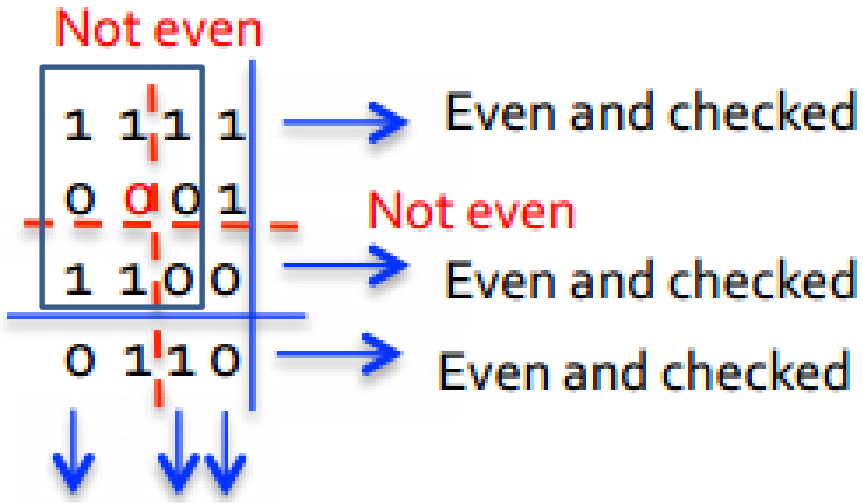
Databit = 9bits
Codeword = 16bits

First in First out
Even parity

Two-Dimensional Parity Bit Checking (2/2)

- If there is an error, two-dimensional parity bit checking can find the position of the error.

At receiver, if there is an error



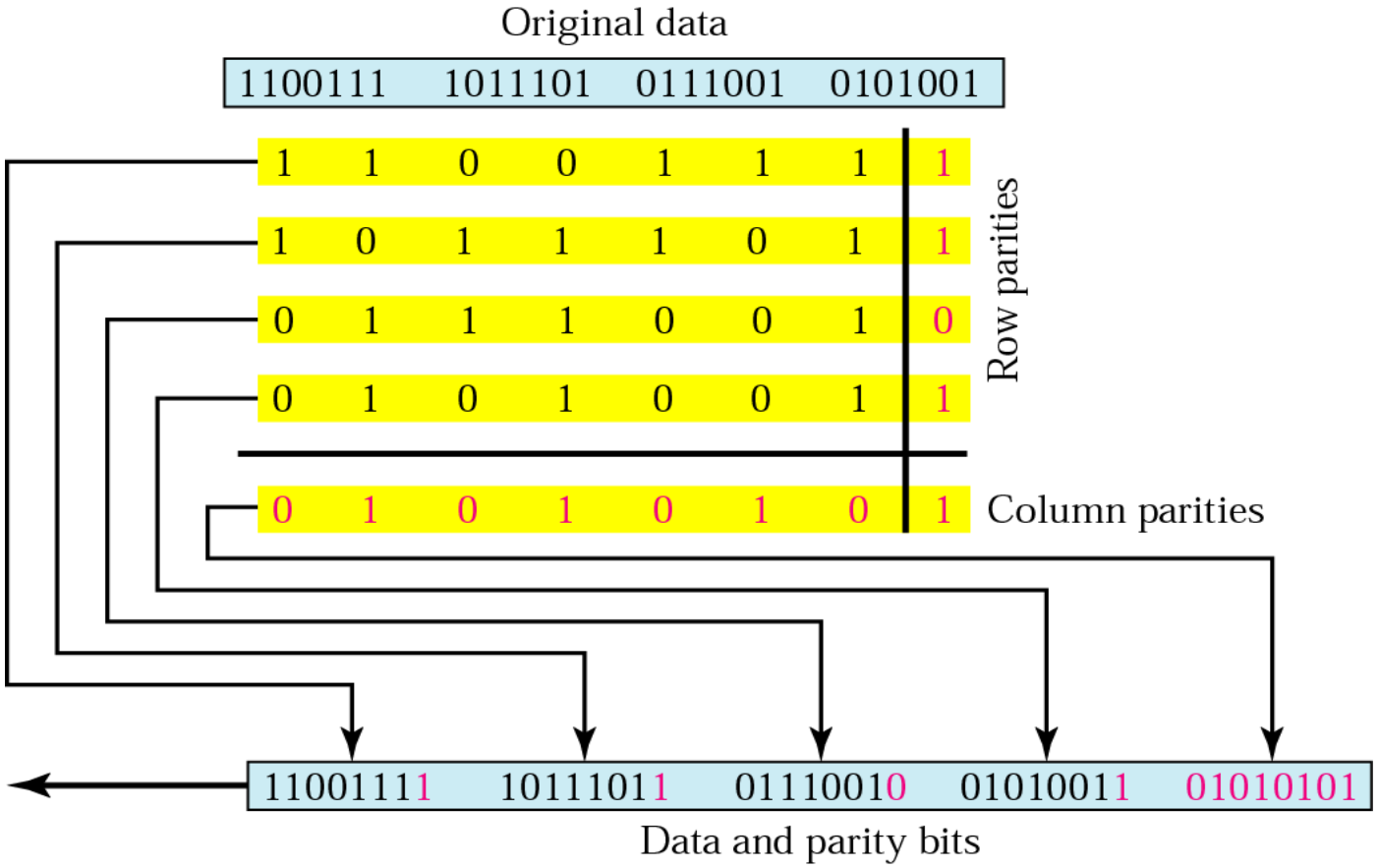
กรณีที่มี 1 error (single error)
เราสามารถใช้ 2-dimensional parity เป็น Error Correction

Two-Dimensional Parity Bit Check

Example 1

- When we transmit the data, we can transmit the data row by row along with the redundancy bits.

Even parity
ส่งจากขวาไปซ้าย
ส่ง bit ซ้ายสุดก่อน



Two-Dimensional Parity Bit Check

Example 2

row parity

$d_{1,1}$	. . .	$d_{1,j}$	$d_{1,j+1}$
$d_{2,1}$	. . .	$d_{2,j}$	$d_{2,j+1}$
. . .	. . .	. . .	. . .
$d_{i,1}$	. . .	$d_{i,j}$	$d_{i,j+1}$
$d_{i+1,1}$	. . .	$d_{i+1,j}$	$d_{i+1,j+1}$

column parity

```
10101|1
11110|0
01110|1
-----
10101|0
```

no errors

```
10101|1
10110|0
01110|1
-----
10101|0
```

parity error

parity error

correctable single bit error

Two-Dimensional Parity Check Limitation

Example 2

- Two-dimensional parity check increases the likelihood of detecting burst error.
- However, one pattern of errors that remain difficult to catch.
  - If 2 bits in one data unit are damaged and two bits in exactly the same positions in another data unit are also damaged, the checker will not detect an error !
  - Example

1	1	1	1
0	1	0	1
1	1	0	0
0	1	1	0

Valid codewords

Error →

1	1	1	1
0	**0**	**1**	1
1	**0**	**1**	0
0	1	1	0

Valid codewords

Fail to detect errors

1	1	1	1
0	**0**	**1**	1
1	1	0	0
0	1	1	0

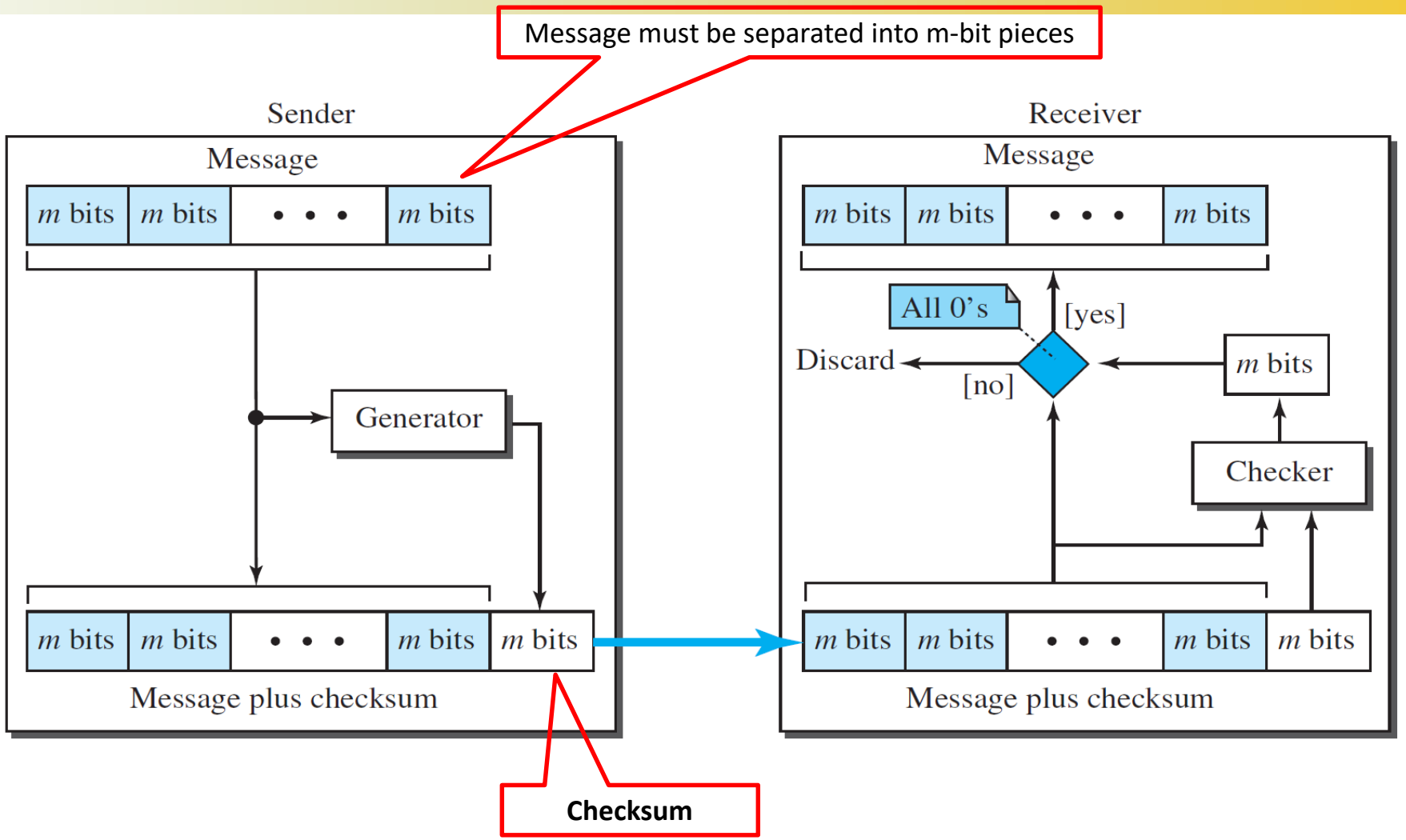
Checksum

- Checksum is an error-detecting technique that can be applied to a message of any length.
- Checksum is based on the summation and one's complement arithmetic.
- In the Internet, the checksum technique is mostly used at the application, network and transport layer rather than the data-link layer (which can work with hardware easily) because it is fast and easy.
  - IP header checksum
  - TCP checksum
- CRC is often used at the data-link layer because it can be implemented easily in the hardware. (We will see CRC after the break)

Checksum Operation (1/2)

- At the source, the message is first divided into m-bit units.
- The generator then creates an extra m-bit unit called the checksum, which is sent with the message.
- At the destination, the checker creates a new checksum from the combination of the message and sent checksum.
- If the new checksum is all 0s, the message is accepted; otherwise, the message is discarded

Checksum Operation (2/2)



Example 1 - Generator

Suppose the following block of 16 bits is to be sent using a checksum of 8 bits.

10101001 00111001

The numbers are added using one's complement

	10101001
	00111001

Sum	11100010
Checksum	00011101

1+1=10
10+1=11

The pattern sent is 10101001 00111001 **00011101**

Example 1 - Checker

Now suppose the receiver receives the pattern sent in Example 1 and there is no error.

10101001 00111001 00011101

When the receiver adds the three sections, it will get all 1s, which, after complementing, is all 0s and shows that there is no error.

	10101001
	00111001
	00011101
Sum	11111111
Complement	**00000000** means that the pattern is OK.

Example 1 – Checker-fail

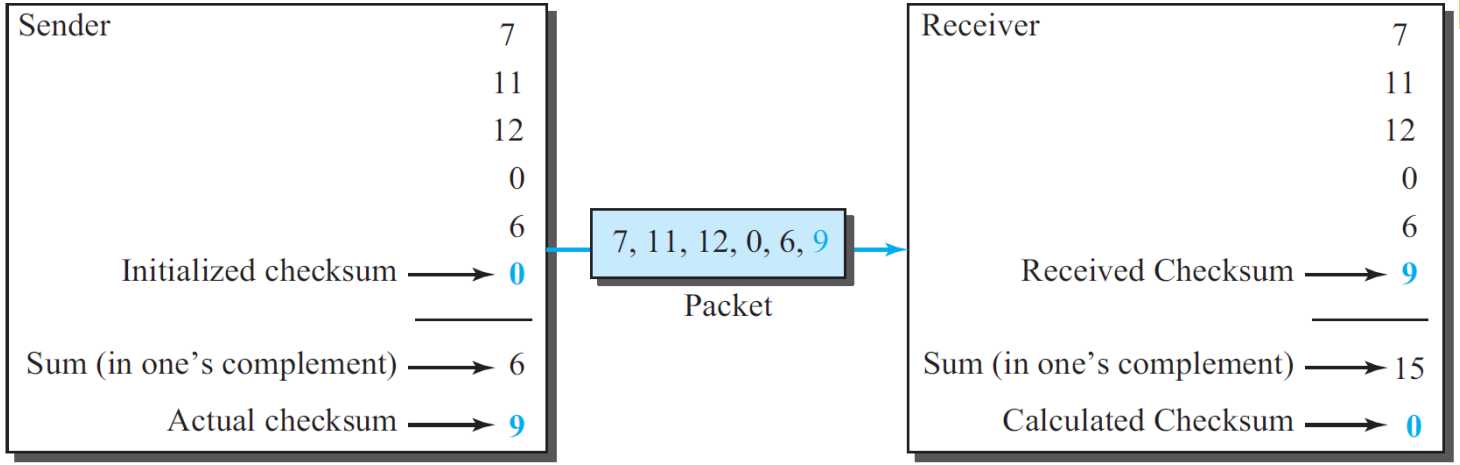
Now suppose there is a **burst error** of length 5 that affects 4 bits.

10101**111 11**111001 00011101

When the receiver adds the three sections, it gets

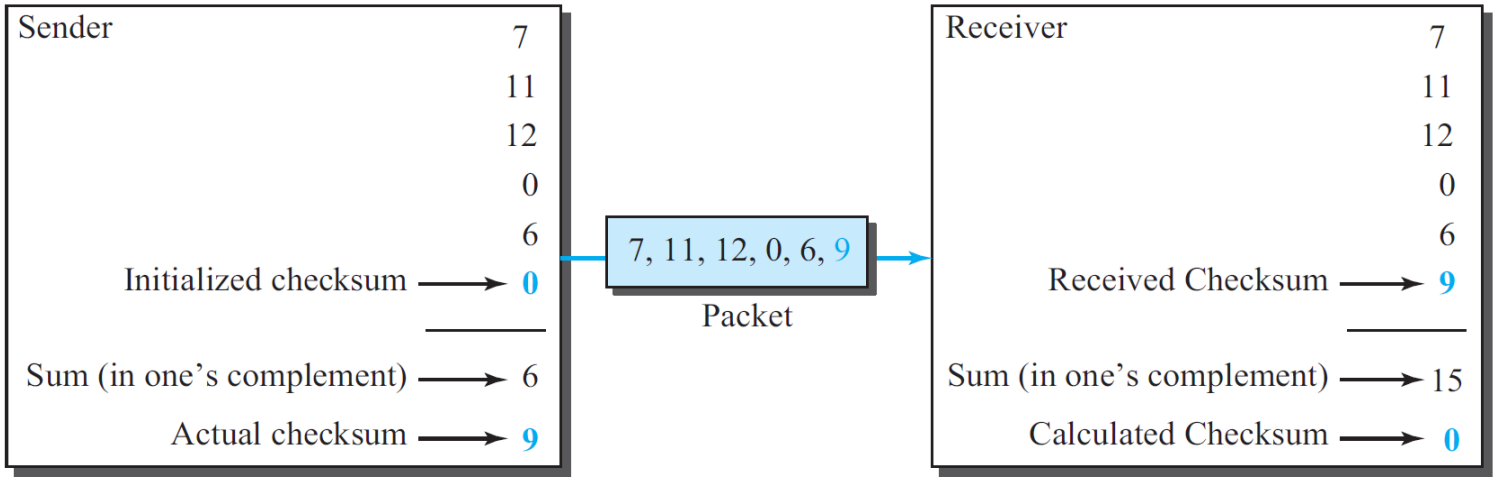
	10101111	
	11111001	
	00011101	
Partial Sum	**1** 11000101	
Carry	**1**	
Sum	11000110	
Complement	**00111001**	**the pattern is corrupted. (syndrome <> 0)**

Example 2: Generator




```
 7      0111    (Word 1)
11      1011    (Word 2)
12      1100    (Word 3)
 0      0000    (Word 4)
 6      0110    (Word 5)
 0      0000    (Initial Checksum @ Sender)
-----------------------
36 0010 0100    (Sum)
-----------------------
        0100    +
        0010
 6      0110    (Wrapped Sum)
-----------------------
 9      1001    (One's Complement of the Wrapped Sum) == Checksum
-----------------------
```

Example 2: Checker




```
 7      0111     (Word 1)
11      1011     (Word 2)
12      1100     (Word 3)
 0      0000     (Word 4)
 6      0110     (Word 5)
 9      1001     (Received Checksum)
 0      0000     (Initialized Checksum)
45      0010    1101     (Sum)

 +      1101     (Wrapped Addition)
        0010
        1111     (One's Complement Sum)
 0      0000     (Checksum – complement of One's Complement Sum)
```

Example 3

Original Data

10011001	11100010	00100100	10000100
1	2	3	4

k=4, m=8

Sender

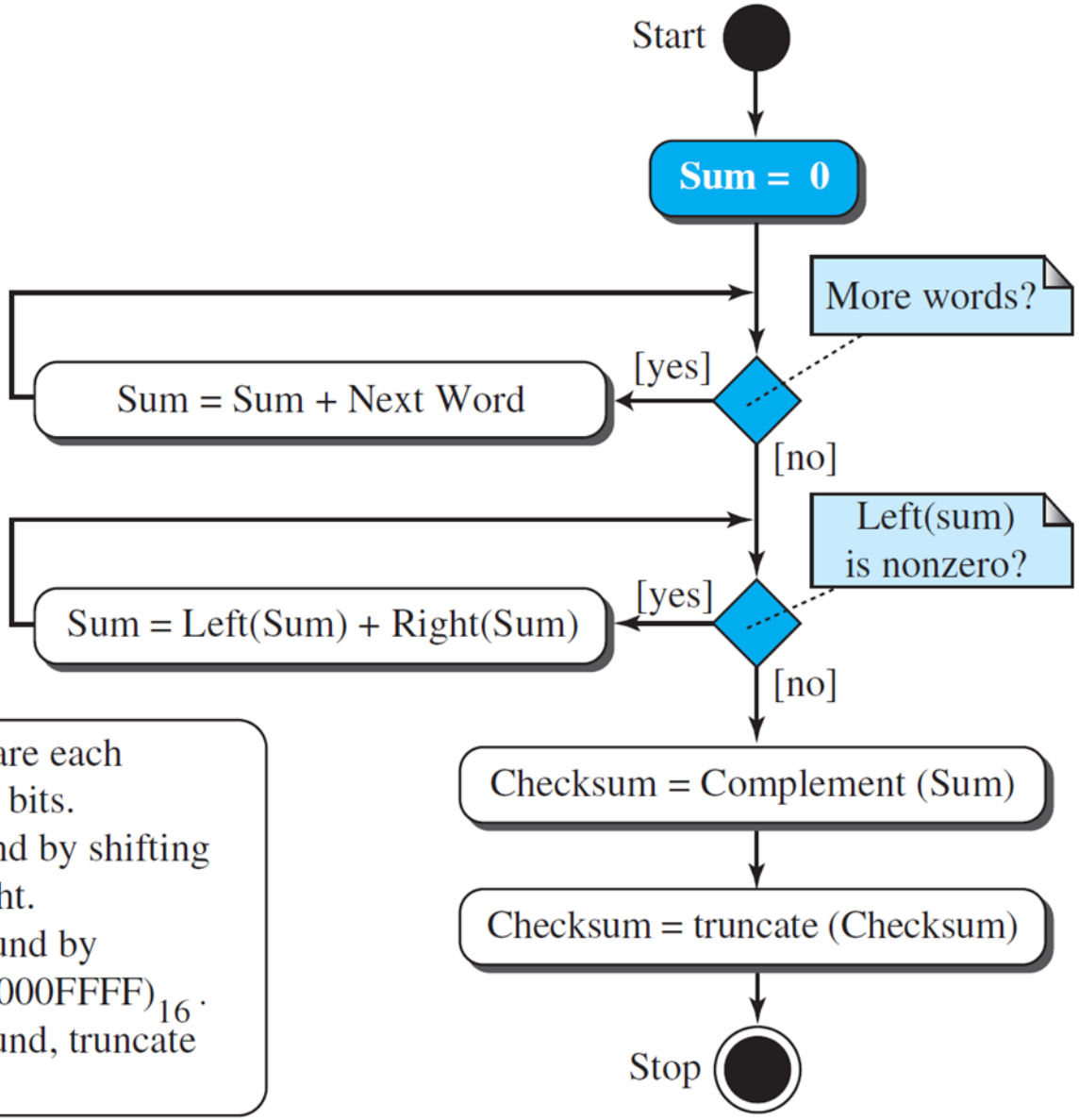
```
1   10011001
2   11100010
    ---------
   (1)01111011
        ------> 1
    ---------
    01111100
3   00100100
    ---------
    10100000
4   10000100
    ---------
   (1)00100100
        ------> 1
    ---------
Sum: 00100101
    ---------
CheckSum: 11011010
```

Reciever

```
1   10011001
2   11100010
    ---------
   (1)01111011
        ------> 1
    ---------
    01111100
3   00100100
    ---------
    10100000
4   10000100
    ---------
   (1)00100100
        ------> 1
    ---------
    00100101
    11011010
    ---------
Sum: 11111111
Complement: 00000000
```

Conclusion: Accept Data

Checksum Generator Algorithm



Notes:

a. Word and Checksum are each 16 bits, but Sum is 32 bits.
b. Left(Sum) can be found by shifting Sum 16 bits to the right.
c. Right(Sum) can be found by ANDing Sum with $(0000FFFF)_{16}$.
d. After Checksum is found, truncate it to 16 bits.

Checksum Performance

- The traditional checksum uses a small number of bits (16) to detect errors in a message of any size (sometimes thousands of bits).

- However, it is not as strong as the CRC in error-checking capability. For example, if the value of one word is incremented and the value of another word is decremented by the same amount, the two errors cannot be detected because the sum and checksum remain the same.

- Also, if the values of several words **are incremented** but the sum and the checksum **do not** change, the errors **are not detected**.

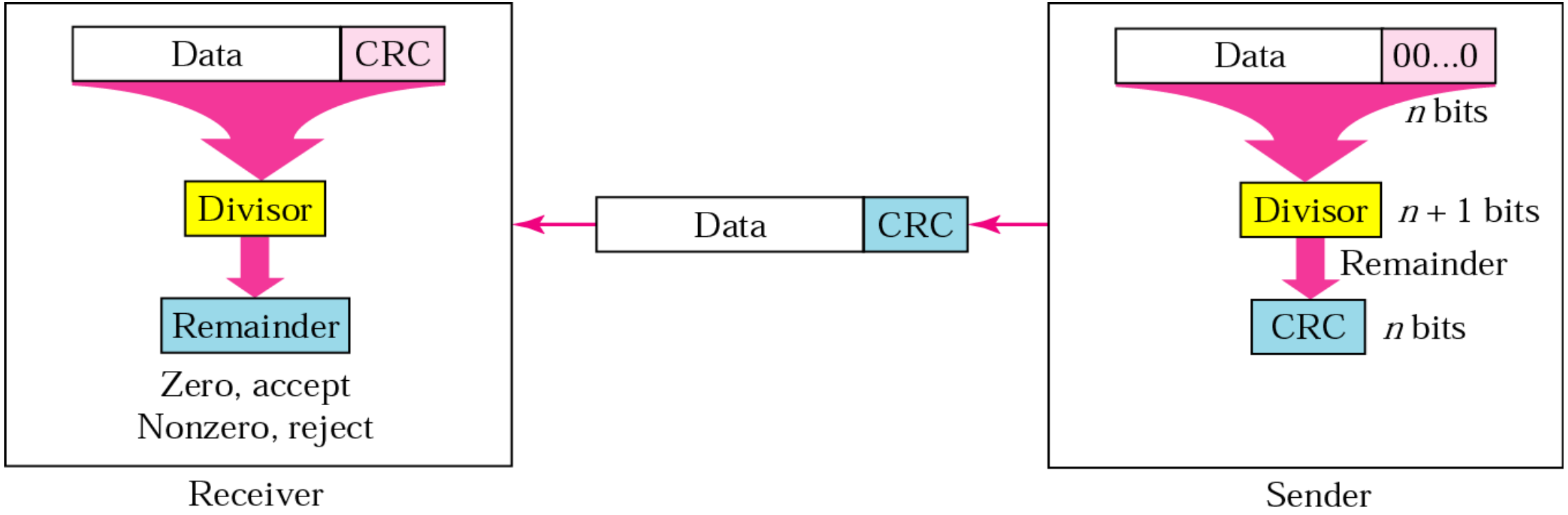
- Fletcher and Adler have proposed some weighted checksums that eliminate the first problem. However, the tendency in the Internet, particularly in designing new protocols, is to replace the checksum with a CRC.

Cyclic Redundancy Check (CRC)

- The most powerful of the redundancy checking techniques
- Unlike the parity check which is based on addition, CRC is based on binary division using a predetermined (previously agreed) divisor (key).
- A sequence of redundant bits called CRC or the CRC remainder is appended to the end of the data unit so that the resulting data unit becomes exactly divisible by a second, predetermined binary number

- **The CRC (redundancy bits)**
  - It must have exactly one less bit than the divisor
  - Appending it to the end of the dataword must make the resulting bit sequence exactly divisible by the divisor (checking method)

CRC Generator and Checker



CRC Binary Division: Generator (Encoder)

Performed at Sender side

Uses modulo-2 division
-1 is treated as 0
Similar to XOR

```
                Quotient
                1 1 1 1 0 1
Divisor 1 1 0 1 ) 1 0 0 1 0 0 0 0 0   <- Data plus extra zeros
                  1 1 0 1
                  -------
                    1 0 0 0
                    1 1 0 1
                    -------
                      1 0 1 0
                      1 1 0 1
                      -------
                        1 1 1 0
                        1 1 0 1
                        -------
                          0 1 1 0
                          0 0 0 0
                          -------
                            1 1 0 0
                            1 1 0 1
                            -------
                              0 0 1   Remainder
```

When the leftmost bit of the remainder is zero, we must use 0000 instead of the original divisor.

CRC Binary Division: Checker (Decoder)

Performed at Sender side

Uses modulo-2 division
-1 is treated as 0
Similar to XOR

```
                Quotient
                1 1 1 1 0 1
Divisor 1 1 0 1 ) 1 0 0 1 0 0 0 0 1
                  1 1 0 1
                  -------
                    1 0 0 0
                    1 1 0 1
                    -------
                      1 0 1 0
                      1 1 0 1
                      -------
                        1 1 1 0
                        1 1 0 1
                        -------
                          0 1 1 0
                          0 0 0 0
                          -------
                            1 1 0 1
                            1 1 0 1
                            -------
                            0 0 0
```

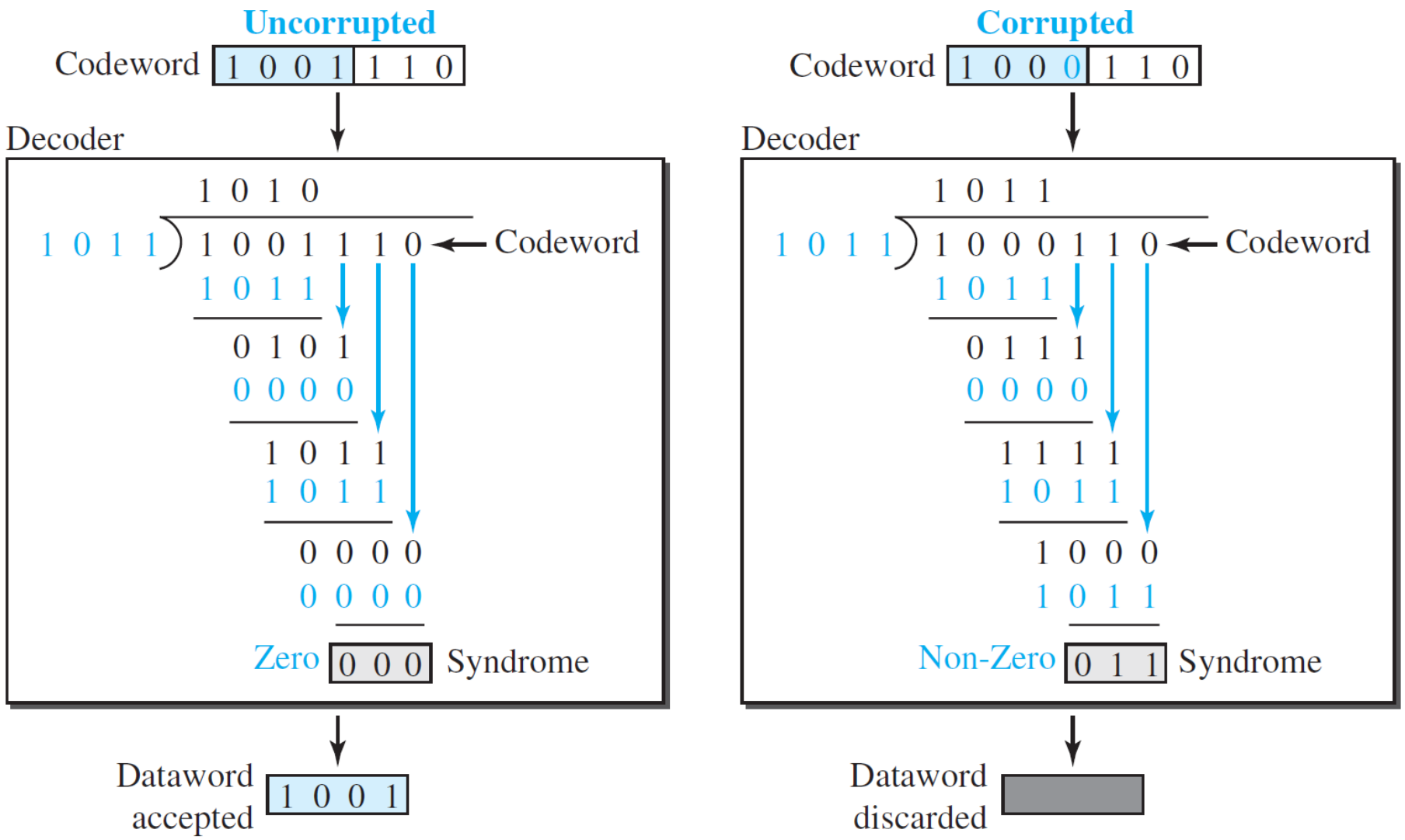
Data plus CRC (Codeword)

When the leftmost bit of the remainder is zero, we must use 0000 instead of the original divisor.

Remainder (syndrome)

CRC Binary Division: Checker (Decoder)

Example



CRC(7,4) Codeword and 4-bit Divisor (1011)

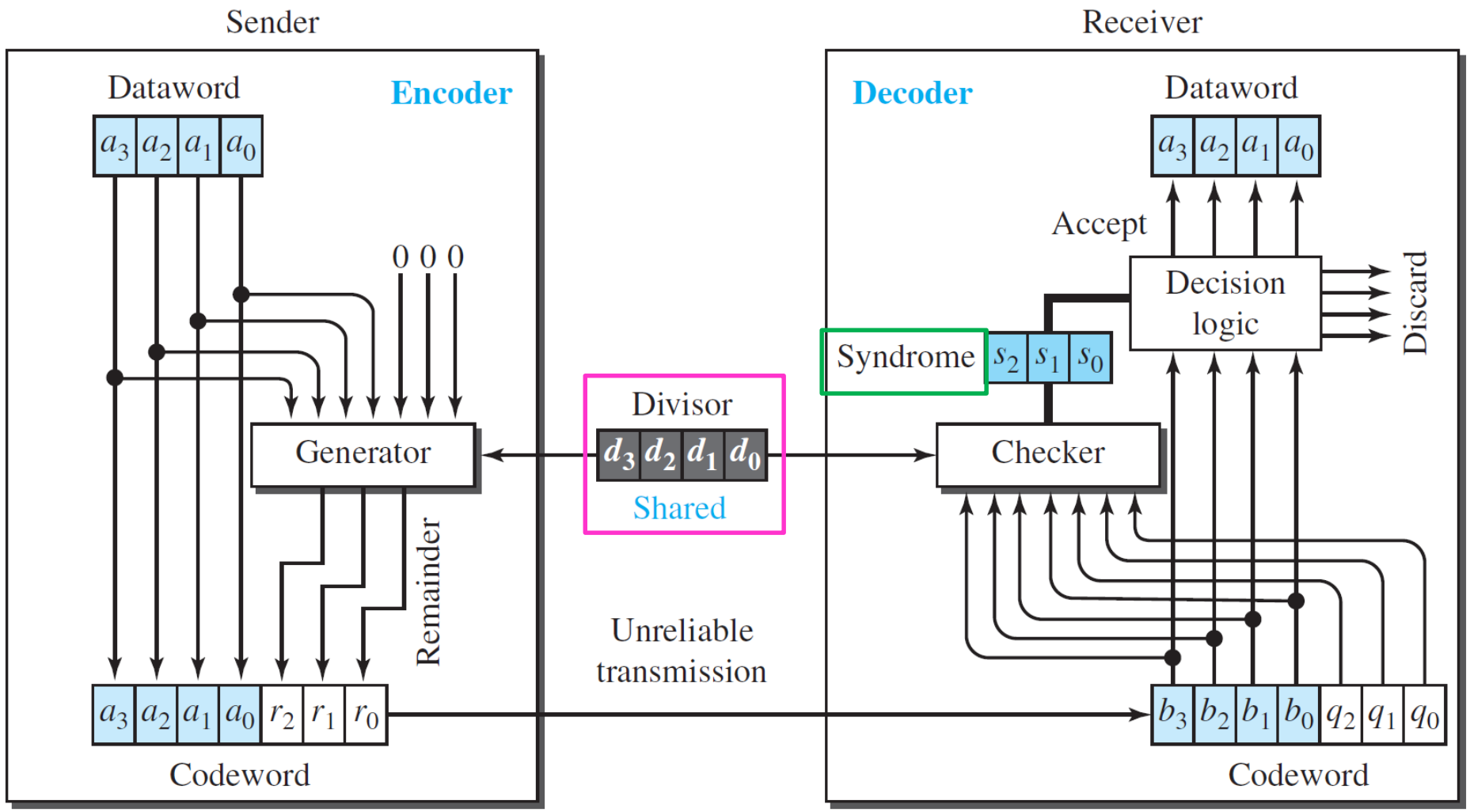
Dataword

ตัวตั้ง(generator): 0010000
ตัวหาร(divisor): 1011
Remainder = CRC=110

Dataword	*Codeword*	*Dataword*	*Codeword*
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

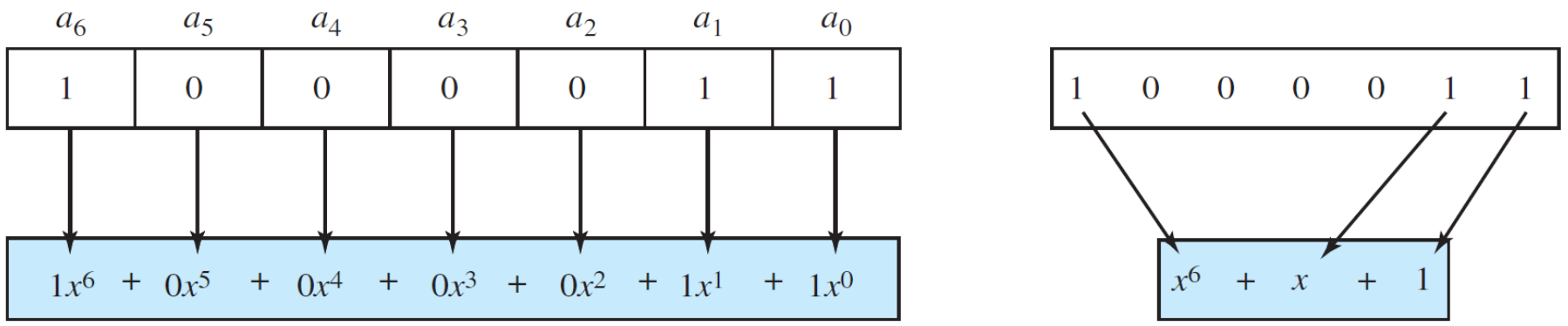
7 bit codeword

CRC(7,4) Codeword and 4-bit Divisor (1011)



CRC Divisors in Polynomials

- Usually, CRC divisors are represented as polynomials.



a. Binary pattern and polynomial

b. Short form

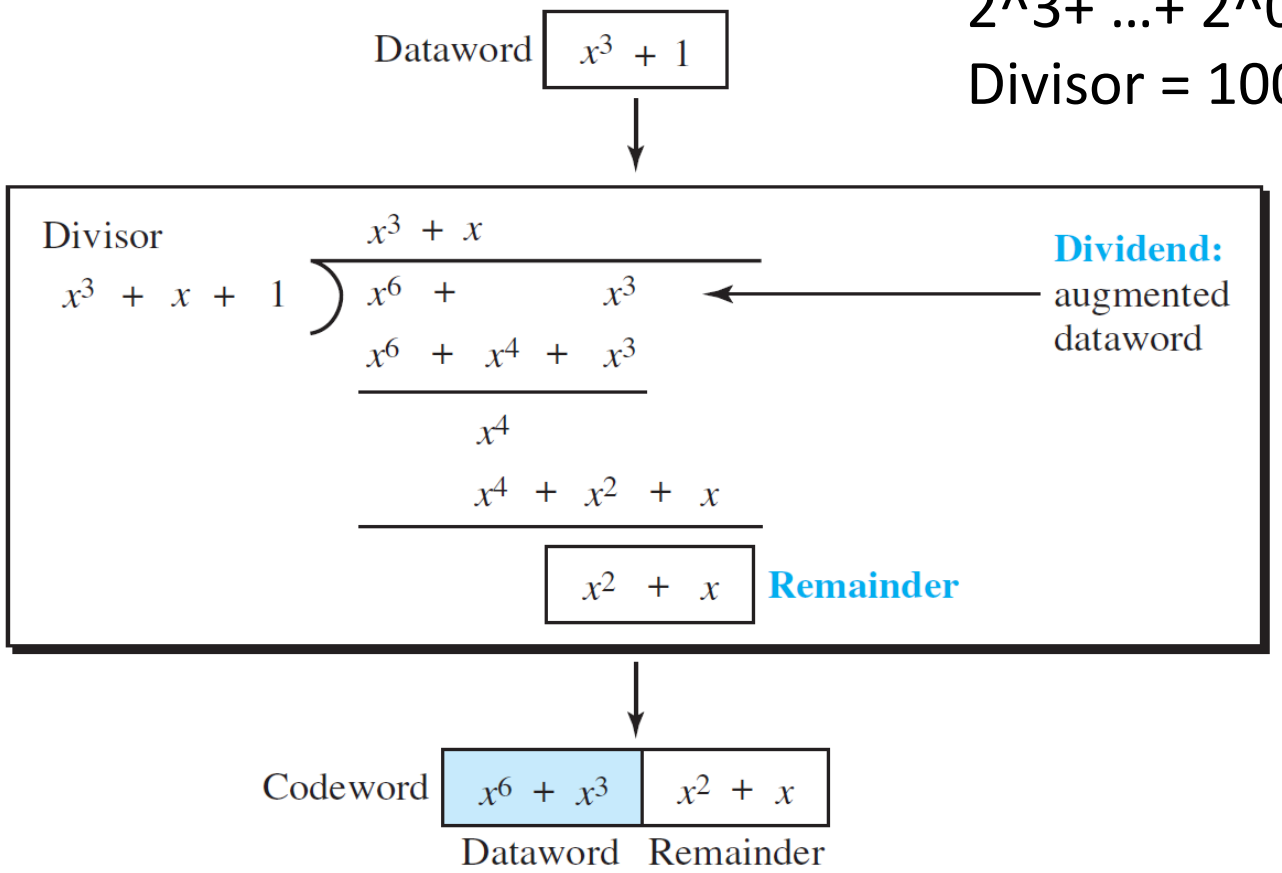
- The divisor in a cyclic code is normally called the generator polynomial or simply **the generator**.

CRC Divisors in Polynomials

- Since CRC divisors are represented as polynomials, the CRC polynomial division will be a bit different from regular polynomial division learnt before.

2^3+ …+ 2^0=

Divisor = 1001



C(x) - Divisor, P(x) - codeword

Polynomial Division

- *Dividing a polynomial with another one of lower degree is similar to normal polynomial division with "subtract" simple (XOR).*

$C(x) = x^3 + x^2 + 1 = 1101$

$P(x) = x^{10} + x^7 + x^5 + x^4 + x^2 + 1 = 10010110101$

```
1101 | 10010110101   <-- P(x)
       1101
        1000
        1101
         1011
         1101
          1101
          1101
            0101   <-- Remainder (lower degree than divisor C(x))
```

C(x)

CRC Divisor and Polynomial Generator Properties

- If the degree of the generator is k,
  - it can detect all burst errors **up to** length k
  - It can detect errors with length > k **with high probability**

- If the generator has more than one term and the coefficient of x^0 is 1, all single-bit errors can be caught.
- If a generator cannot divide $x^t + 1$(t between 0 and n-1), then all isolated double errors can be detected.
- A generator that contains a factor of (x+1) can detect all odd-numbered errors.

- Hence, a good polynomial generator needs to have the following characteristics
  - It should have at least two terms.
  - The coefficient of x^0 should be 1
  - It should not divide $x^t + 1$ for t between 2 and n-1
  - It should have the factor (x + 1)

CRC Divisor Standards

- CRC divisors are standardized in order to help manufacturers produce network equipment with error checking capability.

Name	*Polynomial*	*Used in*
CRC-8	$x^8 + x^2 + x + 1$ **100000111**	ATM header
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$ **11000110101**	ATM AAL
CRC-16	$x^{16} + x^{12} + x^5 + 1$ **10001000000100001**	HDLC
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$ **100000100110000010001110110110111**	LANs

CRC Hardware Implementation
https://www.youtube.com/watch?v=sNkERQlK8j8
https://www.youtube.com/watch?v=tdT1CA54n-k
https://www.youtube.com/watch?v=uc9rMsWsRFo

Error Correction

Some slides from
- ITCS323/2021s1
- NETACAD Network Essentials/ITN7.02

Error Correction

- Normally, when an error is detected, the receiver must send a request for retransmission to the sender (reliable transmission mode only).

- However, there are cases where the receiver can just correct the error by itself using the redundancy sent by the sender.

- Popular error correction methods:
  - Hamming code
  - Interleaving

Hamming Code

- Hamming code allows us to correct a single bit error by identifying the position of the erroneous bit.
- Hamming code uses simple parity check code to place redundant bits at every 2^k position of the codeword.
- Each redundant bit is responsible for checking bits at designated positions.

d7	d6	d5	r8	d4	d3	d2	r4	d1	r2	r1
11	10	9	8	7	6	5	4	3	2	1

r1 is the even redundancy bit for the following bits in the codeword: Bits at position {1,3,5,7,9,11,…}
r2 is the even redundancy bit for the following bits in the codeword: Bits at position {2,3,6,7,10,11,…}
r4 is the even redundancy bit for the following bits in the codeword: Bits at position {4,5,6,7,12,13,14,15,…}
R8 is the even redundancy bit for the following bits in the codeword: Bits at position {8,9,10,11,12,13,14,15…}

d7	d6	d5	r8	d4	d3	d2	r4	d1	r2	**r1**
11	10	9	8	7	6	5	4	3	2	1

d7	d6	d5	r8	d4	d3	d2	**r4**	d1	r2	r1
11	10	9	8	7	6	5	4	3	2	1

d7	d6	d5	r8	d4	d3	d2	r4	d1	**r2**	r1
11	10	9	8	7	6	5	4	3	2	1

d7	d6	d5	**r8**	d4	d3	d2	r4	d1	r2	r1
11	10	9	8	7	6	5	4	3	2	1

Hamming Code Operation

- Generally, given that the codeword starts at position 1, the redundancy bit at position k of the codeword is as follows:
- p(k) = even parity check bit of {d(k:2k-1), d(3k:4k-1), d(5k, 6k-1), d(7k:8k-1),…}

Bit position		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	
Encoded data bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8	d9	d10	d11	p16	d12	d13	d14	d15	
Parity bit coverage	p1	X		X		X		X		X		X		X		X		X		X		
	p2		X	X			X	X			X	X			X	X			X	X		…
	p4				X	X	X	X					X	X	X	X					X	
	p8								X	X	X	X	X	X	X	X						
	p16																X	X	X	X	X	

Example (1/2)

- Dataword: **1001101**

d7	d6	d5	r8	d4	d3	d2	r4	d1	r2	r1
11	10	9	8	7	6	5	4	3	2	1
1	**0**	**0**		**1**	**1**	**0**		**1**		

- r1

d7	d6	d5	r8	d4	d3	d2	r4	d1	r2	r1
11	10	9	8	7	6	5	4	3	2	1
1	**0**	**0**		**1**	**1**	**0**		**1**		**1**

Even parity bit

- r2

d7	d6	d5	r8	d4	d3	d2	r4	d1	r2	r1
11	10	9	8	7	6	5	4	3	2	1
1	**0**	**0**		**1**	**1**	**0**		**1**	**0**	**1**

Even parity bit

Example (2/2)

- Dataword: **1001101**

d7	d6	d5	r8	d4	d3	d2	r4	d1	r2	r1
11	10	9	8	7	6	5	4	3	2	1
1	**0**	**0**		**1**	**1**	**0**		**1**		

- r4

d7	d6	d5	r8	d4	d3	d2	r4	d1	r2	r1
11	10	9	8	7	6	5	4	3	2	1
1	**0**	**0**		**1**	**1**	**0**	**0**	**1**	**0**	**1**

Even parity bit

- r8

d7	d6	d5	r8	d4	d3	d2	r4	d1	r2	r1
11	10	9	8	7	6	5	4	3	2	1
1	**0**	**0**	**1**	**1**	**1**	**0**	**0**	**1**	**0**	**1**

Even parity bit

So the codeword is **10011100101**

Example: One Bit Error

The Received codeword: 10010100101

The sent codeword: 10011100101

Position	d7	d6	d5	r8	d4	d3	d2	r4	d1	r2	r1
Valid codeword	1	0	0	1	1	1	0	0	1	0	1
Received codeword	1	0	0	1	0	1	0	0	1	0	1

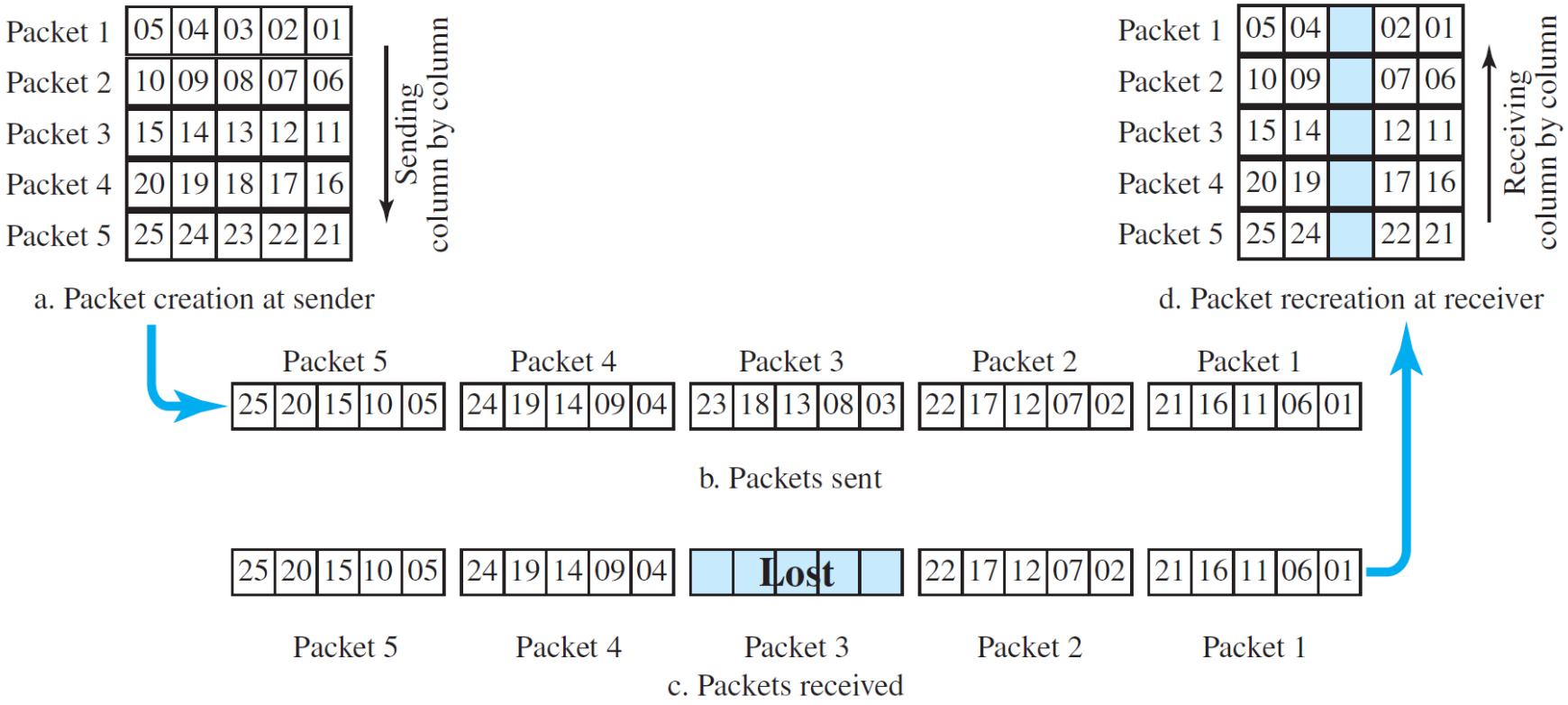
Check r1 = 1 = d7,d5,d4,d2,d1,r1 = 1,0,0,0,1,1
Check r2 = 1 = d7,d6,d4,d3,d1,r2 = 1,0,0,1,1,0
Check r4 = 1 = d4,d3,d2,r4 = 0,1,0,0
Check r8 = 0 = d7,d6,d5,r8 = 1,0,0,1

Check r8											
	1	0	0	1	0	1	0	0	1	1	1

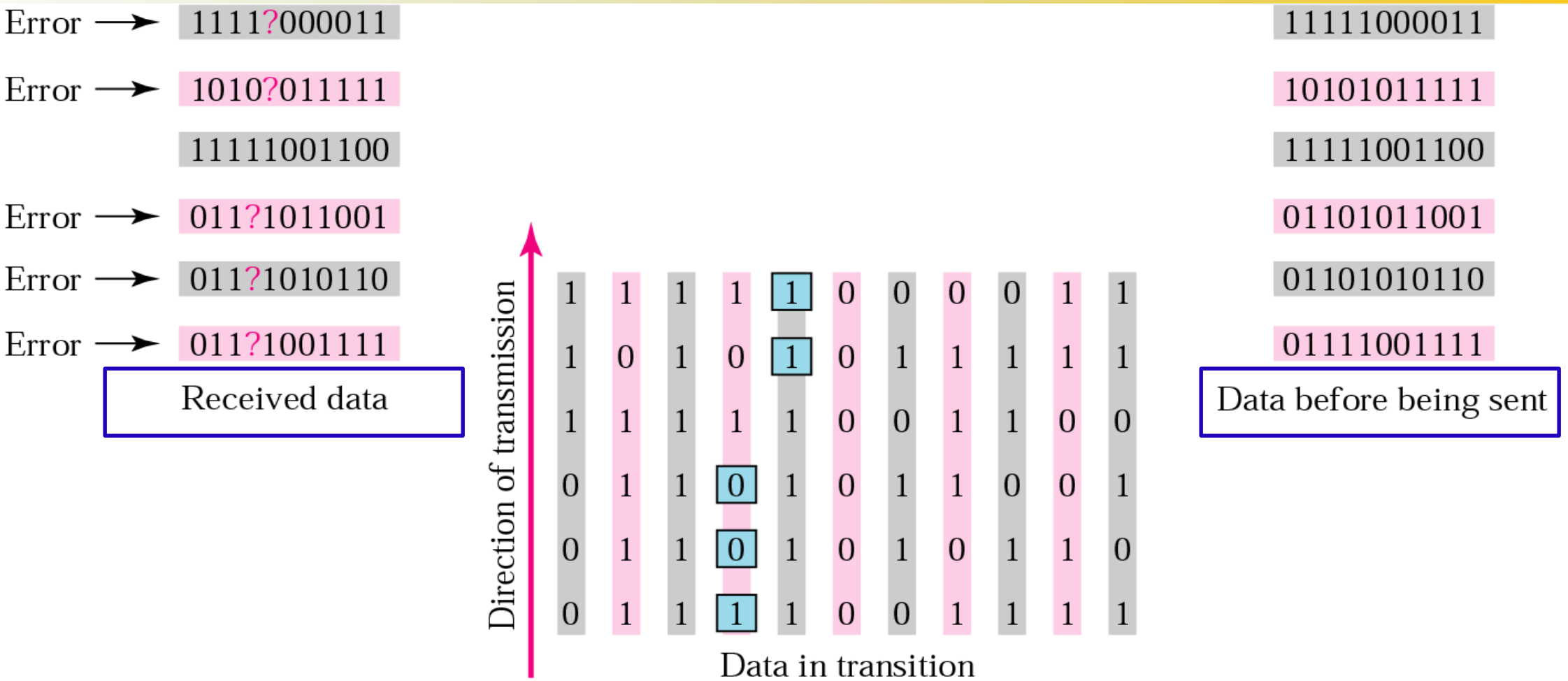
Location of error = $(r8\ r4\ r2\ r1)_2 = (0111)_2 = 7$

Interleaving

- Normally, when a codeword is found to be in error, the entire codeword is discarded.
- However, if we shuffle (interleave) the loss of a codeword can be seen as a single bit error of many codewords.
- The single bit error of codewords can then be fixed using hamming code.



Burst error correction example when using Interleaving



The trick is to let the burst error corrupt only 1 bit of each unit